

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2014

Huấn luyện viên: Hu Weidong

China Computer Federation, 2014

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014

IOI China candidate team papers / National training team 2014 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2014. Phần mục lục và đầu sách có lớp văn bản đọc được, nhưng thân bài dùng ảnh xạ phông chữ khiến kết quả trích xuất bị biến dạng. Bản dịch này chuyển ngữ phần đầu sách, mục lục và năm bài đầu tiên bằng cách kết hợp mục lục trích xuất được với OCR từ các trang đã render.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014*. Huấn luyện viên ghi trên bìa là Hu Weidong. Đơn vị xuất bản là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Báo cáo ra đề <i>MSS</i>	Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh, Sơn Đông
9	Báo cáo ra đề <i>Matrix</i>	Yu Xingjiang, Trường Trường Quận, Trường Sa, Hồ Nam
19	Đa giác biến đổi	Dong Honghua, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
35	Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị	Cen Ruoxu, Trường Trung học Trần Hải, Chiết Giang
45	Bàn về phụ thuộc tuyến tính	Kuang Zhengfei, Trường Yali, Trường Sa, Hồ Nam
57	Một số nghiên cứu về cây khung tích nhỏ nhất ba chiều	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
65	Bàn về bài toán xâu con palindrome	Xu Yi, Trường Trung học cao cấp Thường Châu, Giang Tô
77	Bàn về ứng dụng phương pháp duy trì mảng nhiều chiều trong bài cấu trúc dữ liệu	Liang Zeyu, Trường Trung học số 1 Hợp Phì, An Huy
91	Ứng dụng cây đoạn trong một lớp bài chia để trị	Xu Yinzhan, Trường Học Quân Hàng Châu, Chiết Giang
105	Thuật toán căn bậc hai: không chỉ là chia khối	Wang Yuetong, Trường Ngoại ngữ Nam Kinh, Giang Tô
115	Bàn về các vấn đề liên quan tới cây động và một số mở rộng đơn giản	Huang Zhi'ao, Trường Yali, Trường Sa, Hồ Nam
137	Ứng dụng thuật toán ngẫu nhiên trong thi tin học	Hu Zecong, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
155	Hiện thực chương trình tính tế: bàn về tối ưu hằng số trong OI	He Qi, Trường Trung học Nam Khai, Thiên Tân
169	Trở về gốc rễ: phép toán bit và ứng dụng	Shen Yang, Trường Trung học Trần Hải, Chiết Giang
195	Một số phương pháp tìm nghiệm tốt thứ k	Yu Dingli, Trường Trung học số 1 Thiệu Hưng, Chiết Giang

3 Báo cáo ra đề *MSS*

Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh

3.1 Mô tả bài toán

3.1.1 Phát biểu

Trên mặt phẳng có N điểm. Mỗi điểm có một trọng số; ban đầu mỗi điểm thuộc một tập khác nhau. Gọi tập hiện tại thứ i là S_i . Cần duy trì một cấu trúc dữ liệu hỗ trợ bốn loại thao tác sau.

- Merge $x\ y$: chuyển mọi điểm trong tập S_x sang tập S_y .
- Split $i\ d\ v$, với $d \in \{0, 1\}$: tạo hai tập mới S_{c+1} và S_{c+2} , trong đó c là số tập hiện có, kể cả các tập rỗng sinh ra trước đó. Sau đó, trong tập S_i , các điểm có tọa độ chiều thứ d không vượt quá v được chuyển vào S_{c+1} , còn các điểm có tọa độ lớn hơn v được chuyển vào S_{c+2} .
- Query i : hỏi giá trị lớn nhất, nhỏ nhất và tổng trọng số của các điểm trong tập S_i .
- Add $i\ d$: cộng d vào trọng số của mọi điểm trong tập S_i .

Để thuận tiện, với mỗi $d \in \{0, 1\}$, mọi điểm trong dữ liệu vào có tọa độ chiều thứ d đôi một khác nhau. Mọi tập được nhắc tới trong các thao tác đều không rỗng.

3.1.2 Dữ liệu vào

Dòng đầu chứa một số nguyên, biểu thị số điểm. Tiếp theo là N dòng, mỗi dòng gồm ba số nguyên lần lượt là hai tọa độ và trọng số của một điểm. Sau đó là một số nguyên Q , biểu thị số thao tác. Tiếp theo là Q dòng, mỗi dòng mô tả một thao tác theo định dạng ở trên.

3.1.3 Dữ liệu ra

Với mỗi thao tác Query, in ra ba số nguyên, lần lượt là trọng số lớn nhất, trọng số nhỏ nhất và tổng trọng số trong tập được hỏi.

3.1.4 Ví dụ

Dữ liệu vào

```
5
4 7 3
18 16 2
14 13 1
10 2 10
3 8 5
11
Merge 2 3
Merge 4 5
Split 2 1 13
Query 4
Query 6
Add 6 2
Query 6
Merge 6 4
Split 6 0 10
```

Query 9
Split 8 0 3

Dữ liệu ra

10 5 15
1 1 1
3 3 3
3 3 3

3.1.5 Quy mô dữ liệu và ràng buộc

Dữ liệu được chia thành năm nhóm:

- nhóm A chiếm 10%: $N, Q \leq 500$;
- nhóm B chiếm 20%: dữ liệu không có thao tác tách;
- nhóm C chiếm 10%: sau khi thao tác tách đầu tiên xuất hiện thì không còn thao tác gộp;
- nhóm D chiếm 20%: các điểm thỏa một quan hệ đặc biệt khiến bài toán suy biến về một chiều;
- nhóm E không có đặc trưng bổ sung.

Ở các nhóm B đến E, $N \leq 50000$, $Q \leq 100000$. Giới hạn thời gian là 3 giây và giới hạn bộ nhớ là 512 MB. Dòng ràng buộc của nhóm D trong bản OCR khá nhiều; phần lời giải phía sau cho thấy nhóm này là trường hợp các điểm có thứ tự một chiều tương thích, nên có thể chuyển mọi thao tác tách về cùng một trục.

3.2 Phân tích bài toán

3.2.1 Các thuật toán đơn giản

Mô phỏng. Với nhóm A, có thể mô phỏng trực tiếp các tập điểm. Độ phức tạp thời gian là $O(NQ)$.

Gộp theo heuristic. Với nhóm B, vì không có thao tác tách, có thể dùng gộp theo kích thước: mỗi tập được duy trì bằng một cây cân bằng. Khi gộp hai tập, ta chèn toàn bộ điểm của tập nhỏ hơn vào cây cân bằng của tập lớn hơn. Mỗi lần một điểm bị chèn lại, kích thước tập chứa nó ít nhất tăng gấp đôi; do đó mỗi điểm bị chèn lại nhiều nhất $O(\log N)$ lần. Độ phức tạp là $O(N \log^2 N + Q)$.

Tách theo heuristic. Với nhóm C, trước hết dùng gộp theo heuristic để xử lý toàn bộ thao tác gộp trước khi thao tác tách đầu tiên xuất hiện. Sau đó, nếu mỗi thao tác tách có thể thực hiện trong thời gian

$$O(T \cdot |\text{tập nhỏ hơn sau khi tách}|),$$

trong đó T không phụ thuộc vào kích thước tập nhỏ hơn, thì phân tích ngược lại giống hệt gộp theo heuristic.

Nếu mỗi lần chỉ tách theo một tọa độ, ta chỉ cần lập cây cân bằng theo thứ tự tọa độ đó. Quay lại bài này, với mỗi tập ta lập hai cây cân bằng, lần lượt sắp theo hoành độ và tung độ. Khi tách, dùng cây theo chiều được hỏi để lấy tập nhỏ hơn; trong cây còn lại, xóa các điểm tương ứng rồi dựng lại. Khi đó $T = O(\log N)$, nên độ phức tạp là $O(N \log^2 N + Q)$.

3.2.2 Thuật toán một chiều

Trong nhóm D, vì các điểm có thứ tự tương thích theo một chiều, mọi phép tách theo tung độ có thể chuyển thành phép tách theo hoành độ. Do đó có thể xem bài toán là duy trì các tập số trên một đường thẳng.

Dựa trên cây cân bằng. Ta tiếp tục thử dùng cây cân bằng để duy trì giá trị trong từng tập. Cây cân bằng hỗ trợ tách trong $O(\log N)$ và gộp trong $O(\log N)$ khi hai tập có khoảng giá trị không giao nhau; vấn đề còn lại là gộp khi các khoảng giá trị có thể xen kẽ nhau.

Thuật toán gộp có thể viết như sau.

```
merge(x, y):
  if x = nil: return y
  if y = nil: return x
  if x.min > y.min: swap(x, y)
  return merge(merge(splitL(x, y.min), y), splitR(x, y.min))
```

Ở đây $\text{merge}(a, b)$ trả về hợp của hai cây cân bằng a, b khi phạm vi giá trị của chúng không giao nhau; $\text{splitL}(a, v)$ trả về cây gồm các phần tử của a có giá trị không vượt quá v ; $\text{splitR}(a, v)$ trả về cây gồm các phần tử của a có giá trị lớn hơn v .

Ta xét độ phức tạp của thuật toán. Cần chứng minh rằng tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log^2 N).$$

Định nghĩa

$$W(X, Y) = \sum_{a \in X, b \in Y} [|\text{rank}(X \cup Y, a) - \text{rank}(X \cup Y, b)| = 1],$$

trong đó $\text{rank}(S, x)$ là thứ hạng theo giá trị của x trong S . Nói không hoàn toàn hình thức, $W(X, Y)$ là số đoạn, trừ đi 1, thu được sau khi gộp X và Y rồi nén các phần tử vốn thuộc cùng một tập thành một đoạn. Một thao tác gộp có chi phí $O(W(X, Y) \log N)$.

Gọi U là hợp của tất cả các tập. Với một phần tử x thuộc tập S , đặt

$$\begin{aligned} g_0(x) &= \text{rank}(U, x) - \text{rank}(U, \text{prev}(S, x)), \\ g_1(x) &= \text{rank}(U, \text{succ}(S, x)) - \text{rank}(U, x), \end{aligned}$$

trong đó $\text{prev}(S, x)$ và $\text{succ}(S, x)$ lần lượt là phần tử trước và sau của x trong tập S ; nếu không tồn tại thì dùng $\min(U)$ hoặc $\max(U)$.

Sau thao tác thứ i , định nghĩa thể năng của cấu trúc dữ liệu là

$$\Phi_i = c \log N \sum_{x \in U} (\log g_0(x) + \log g_1(x)),$$

với c là hằng số sẽ chọn sau. Ban đầu $\Phi_0 = cN \log^2 N$.

Một thao tác tách có chi phí thực $O(\log N)$. Vì chỉ có một giá trị g_0 và một giá trị g_1 tăng, phần tăng thể năng không vượt quá $2c \log^2 N$.

Xét thao tác gộp X và Y . Đặt

$$\begin{aligned} P_X &= \{x \in X \mid \text{prev}(X, x) \neq \text{prev}(X \cup Y, x)\} \cup \{\min(X)\}, \\ S_X &= \{x \in X \mid \text{succ}(X, x) \neq \text{succ}(X \cup Y, x)\} \cup \{\max(X)\}. \end{aligned}$$

Sau khi gộp X, Y và nén các điểm vốn thuộc cùng một tập thành đoạn, P_X chứa mọi đầu trái của các đoạn đến từ X , còn S_X chứa mọi đầu phải. Định nghĩa tương tự P_Y, S_Y . Khi đó

$$|P_X| + |P_Y| = W(X, Y) + 1.$$

Với các cặp biên kề nhau sau khi gộp, trong hai đại lượng g_0 và g_1 sẽ có một đại lượng giảm ít nhất còn một nửa giá trị cũ. Vì có $W(X, Y) - 1$ cặp như vậy, phần biến thiên thế năng do gộp không vượt quá

$$-c(W(X, Y) - 1) \log N.$$

Chọn c đủ lớn, tổng chi phí mọi thao tác được chặn bởi

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} O(\log N + c \log^2 N) \\ + \sum_{i=1}^{Q_2} (-c(W(X_i, Y_i) - 1) \log N + O(W(X_i, Y_i) \log N)) \\ = O((Q_1 + N) \log^2 N). \end{aligned}$$

Dựa trên cây đoạn. Một cách khác là dùng cây đoạn động để duy trì từng tập. Cây đoạn động hỗ trợ gộp; sau khi rời rạc hóa tọa độ, thao tác tách tốn $O(\log N)$. Cần chứng minh tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log N).$$

Gọi $\text{size}(T)$ là số nút của cây đoạn động biểu diễn tập T . Chi phí gộp hai tập X, Y là

$$O(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Định nghĩa thế năng sau thao tác thứ i :

$$\Phi_i = c \sum_{S \in \mathcal{S}_i} \text{size}(S),$$

trong đó $\mathcal{S}_i$ là họ các tập hiện tại. Một thao tác tách có chi phí thực $O(\log N)$ và chỉ làm tăng thế năng thêm $O(c \log N)$. Một thao tác gộp X, Y làm thế năng giảm

$$c(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Thế năng ban đầu là $cN \log N$. Chọn c đủ lớn, tổng chi phí không vượt quá

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} (O(\log N) + c \log N) \\ + \sum_{i=1}^{Q_2} (O(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i)) \\ - c(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i))) \\ = O((N + Q) \log N). \end{aligned}$$

Từ đó nhóm D được giải với độ phức tạp $O((N + Q) \log N)$.

Thuật toán thay thế. Đây là một bài toán kinh điển. Tài liệu tham khảo [3] đưa ra thuật toán trung bình $O(\log N)$ dựa trên *biased skip list*, nhưng cả phân tích lẫn hiện thực đều phức tạp hơn; độc giả quan tâm có thể tự tham khảo.

3.2.3 Thuật toán tổng quát

Xét phạm vi áp dụng của thuật toán một chiều. Định nghĩa quan hệ thứ tự riêng phần trên tập điểm:

$$(x_1, y_1) < (x_2, y_2) \quad \text{khi và chỉ khi} \quad x_1 < x_2, y_1 < y_2.$$

Nếu hợp của mọi tập hiện tại U , theo quan hệ này, là một chuỗi, thì thuật toán một chiều áp dụng được: do y tăng đơn điệu theo x , mọi phép chia theo tọa độ y đều có thể chuyển thành phép chia theo tọa độ x . Tương tự, khi U là một phản chuỗi, tức y giảm đơn điệu theo x , thuật toán một chiều cũng áp dụng được.

Trong trường hợp tổng quát, chia U thành một số tập con $U_1, \dots, U_K$ sao cho theo thứ tự riêng phần ở trên, mỗi U_i là một chuỗi hoặc phản chuỗi. Với mỗi U_i , xét giao của nó với từng tập hiện tại:

$$\{S \cap U_i \mid S \in \mathcal{S}\}.$$

Khi đó các thao tác gộp, tách và hỏi đều có thể xử lý độc lập trên từng U_i , và mỗi U_i được duy trì bằng thuật toán một chiều.

Độ phức tạp của cách này là $O((N + Q)K \log N)$. Còn cần chứng minh $K = O(\sqrt{N})$.

Bổ đề 1. Trong một tập có thứ tự riêng phần P kích thước N , hoặc tồn tại một chuỗi kích thước $\sqrt{N}$, hoặc tồn tại một phản chuỗi kích thước $\sqrt{N}$.

Chứng minh. Giả sử P không chứa chuỗi độ dài L . Theo định lý Dilworth, P có thể được chia thành không quá $L - 1$ phản chuỗi. Vì vậy tồn tại một phản chuỗi có kích thước ít nhất $N/(L - 1)$. Chọn $L = \lceil \sqrt{N} \rceil$ là đủ.

Bổ đề 2. Một tập có thứ tự riêng phần U kích thước N có thể được chia thành $O(\sqrt{N})$ chuỗi hoặc phản chuỗi.

Chứng minh. Chia đệ quy: mỗi lần xóa khỏi tập còn lại một chuỗi dài nhất hoặc một phản chuỗi dài nhất. Nếu tập còn lại có kích thước n , theo bổ đề 1 có thể xóa ít nhất $\sqrt{n}$ phần tử. Số phần trong phép chia thỏa

$$T(N) = T(N - \sqrt{N}) + 1 < T(N/2) + O(\sqrt{N}) = O(\sqrt{N}).$$

Như vậy bài toán được giải với độ phức tạp

$$O((N + Q)\sqrt{N} \log N).$$

3.3 Kết luận

Kiến thức dùng trong bài chỉ gồm cấu trúc dữ liệu cơ bản và phân tích khấu hao, đều thuộc phạm vi của thí sinh NOI. Tuy nhiên, việc tìm ra thuật toán tổng quát đòi hỏi năng lực phân tích khá tốt.

Đối với 40 điểm cuối, tuy còn tồn tại những cách giải khác, tác giả cho rằng cách chia tập có thứ tự riêng phần được trình bày ở đây thú vị và giàu tính gợi mở hơn. Khi đối tượng trong bài toán thỏa một quan hệ thứ tự riêng phần khác, chẳng hạn quan hệ bao hàm giữa các đoạn, những phương pháp tương tự đáng được cân nhắc.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Richard A. Brualdi, *Introductory Combinatorics*, 5th ed., Prentice Hall.
3. John Iacono, Ozgur Ozkan, “Mergeable Dictionaries”.

4 Báo cáo ra đề *Matrix*

Yu Xingjiang, Trường Trường Quận, Trường Sa

4.1 Đề bài

4.1.1 Mô tả

Cho một ma trận số thực không âm A kích thước $N \times N$. Cần tìm một ma trận số thực B kích thước $N \times N$ sao cho

$$\sum_{k=1}^N B_{i,k} + \sum_{k=1}^N B_{k,j} \geq A_{i,j}$$

với mọi i, j . Nói cách khác, với mỗi ô (i, j) , tổng các phần tử trên hàng i của B cộng với tổng các phần tử trên cột j của B phải không nhỏ hơn $A_{i,j}$. Trong điều kiện đó, hãy tối thiểu hóa tổng mọi phần tử của B .

4.1.2 Dữ liệu vào

Có hai nhiệm vụ, chi tiết xem ở phần phạm vi dữ liệu. Với mỗi nhiệm vụ, dòng đầu chứa một số nguyên N . Tiếp theo là N dòng, mỗi dòng chứa N số thực không âm, mô tả ma trận A đã cho.

4.1.3 Dữ liệu ra

In hai dòng. Mỗi dòng in một số thực không âm giữ lại 4 chữ số sau dấu thập phân, lần lượt là đáp án của hai nhiệm vụ.

4.1.4 Ví dụ

Dữ liệu vào

```
3
1 7 3
3 2 1
5 4 1
2
1 1
1 1
```

Dữ liệu ra

```
6.5000
1.0000
```


4.1.5 Nhiệm vụ con và quy ước

- Task0: mọi phần tử trong A bằng nhau.
- Task1: trong cả nhiệm vụ một và nhiệm vụ hai, $N \leq 50$.
- Task2: các phần tử của A là ngẫu nhiên.
- Task3: trong nhiệm vụ hai, $N \leq 50$.
- Task4: trong nhiệm vụ một, $N \leq 300$; trong nhiệm vụ hai, $N \leq 80$.

Ở nhiệm vụ một, ma trận B cần dựng không có ràng buộc nào khác. Ở nhiệm vụ hai, mọi phần tử của B phải không âm. Mọi số thực trong dữ liệu vào giữ lại 3 chữ số sau dấu thập phân và có trị tuyệt đối không vượt quá 1000.

Giới hạn thời gian là 3 giây, giới hạn bộ nhớ là 512 MB.

4.2 Ý tưởng giải

4.2.1 Task0

Dễ thấy khi mọi phần tử của A bằng nhau, mọi phần tử trong một nghiệm tối ưu của B cũng có thể chọn bằng nhau. Do đó có thể tính trực tiếp đáp án bằng tay.

4.2.2 Lập mô hình quy hoạch tuyến tính

Theo đề bài, mô hình quy hoạch tuyến tính trực tiếp là

$$\begin{cases} \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{1,j} \geq A_{1,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{1,j} \geq A_{1,2}, \\ \dots \\ \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{2,j} \geq A_{2,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{2,j} \geq A_{2,2}, \\ \dots \\ \sum_{i=1}^N B_{i,N} + \sum_{j=1}^N B_{N,j} \geq A_{N,N}, \end{cases} \quad \min z = \sum_{i=1}^N \sum_{j=1}^N B_{i,j}.$$

Cách lập này có $O(N^2)$ biến, cần tối ưu thêm. Đề bài thực ra đã gợi ý rằng có thể dùng tổng hàng và tổng cột làm biến, nên ta thử hướng đó. Gọi R_i là tổng trọng số của hàng i , C_j là tổng trọng số của cột j . Khi đó có một mô hình mới:

$$\begin{cases} R_1 + C_1 \geq A_{1,1}, \\ R_1 + C_2 \geq A_{1,2}, \\ \dots \\ R_2 + C_1 \geq A_{2,1}, \\ R_2 + C_2 \geq A_{2,2}, \\ \dots \\ R_N + C_N \geq A_{N,N}, \\ \sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0. \end{cases}$$

Trong bài gốc, hàm mục tiêu của mô hình này được viết là

$$\min z = \sum_{i=1}^N R_i + \sum_{j=1}^N C_j.$$

Vì luôn có $\sum R_i = \sum C_j$, việc tối thiểu hóa biểu thức trên tương đương với tối thiểu hóa tổng phần tử của B , chỉ khác một hệ số hằng.

So với mô hình ban đầu, số biến giảm xuống còn $O(N)$. Tuy nhiên, còn một câu hỏi: có chắc mọi nghiệm của mô hình tổng hàng/tổng cột đều tương ứng với một ma trận B hay không?

Với nhiệm vụ một, do các phần tử của B có thể nhận giá trị tùy ý, nghiệm tương ứng luôn tồn tại.

Với nhiệm vụ hai, xét các ràng buộc hiện tại:

$$\sum_{i=1}^N R_i = \sum_{j=1}^N C_j, \quad R_i \geq 0, \quad C_j \geq 0.$$

Lấy tùy ý một R_i , rồi rút một phần giá trị từ các C_j để đưa vào hàng i . Do $\sum_j C_j \geq R_i$, luôn tồn tại cách rút sao cho sau đó mọi C_j vẫn không âm. Sau khi xử lý xong R_i , bài toán còn lại có dạng ràng buộc không đổi. Vì vậy chắc chắn tồn tại một ma trận không âm B tương ứng với nghiệm của mô hình trên.

4.2.3 Task1

Khi cả hai nhiệm vụ đều có $N \leq 50$, có thể dùng đơn hình hoặc thuật toán tổng quát tương tự để giải trực tiếp quy hoạch tuyến tính.

Với nhiệm vụ hai, mô hình đúng là dạng chuẩn của quy hoạch tuyến tính. Với nhiệm vụ một, cần tách mỗi R_i thành $R_i^a - R_i^b$, trong đó $R_i^a, R_i^b \geq 0$; với C_j làm tương tự. Trên dữ liệu ngẫu nhiên, đơn hình chạy rất tốt, thậm chí có thể nhanh hơn lời giải chính. Nếu hiện thực tốt, ví dụ thêm cắt tĩa bằng danh sách liên kết, cách này cũng có thể qua được dữ liệu của Task3. Độ phức tạp thời gian: không xác định.

4.2.4 Nguyên lý đối ngẫu

Quy hoạch tuyến tính gốc dường như không dễ xử lý tiếp, nên ta thử chuyển sang bài toán đối ngẫu.

Nhiệm vụ một. Có thể dự đoán rằng trị tuyệt đối của các phần tử trong ma trận đáp án không quá lớn. Nếu cộng cho mỗi phần tử một hằng số rất lớn, rồi dùng kết luận của Task0, nhiệm vụ một sẽ được chuyển thành nhiệm vụ hai. Dĩ nhiên làm trực tiếp như vậy chắc chắn sẽ quá thời gian, nên ta dùng thẳng nguyên lý đối ngẫu.

Bài toán đối ngẫu thu được có dạng

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P = 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P = 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P = 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P = 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P = 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Nếu không có biến P , đây là mô hình luồng chi phí lớn nhất rất quen thuộc trên đồ thị hai phía. Chú ý rằng mọi ràng buộc đều lấy dấu bằng. Điều này có nghĩa là trong mạng chi phí đó, tổng dung lượng các cạnh rời khỏi S bằng tổng dung lượng các cạnh đi vào T ; từ đó suy ra ngay $P = 0$. Phần còn lại chỉ là một bài toán luồng chi phí đơn giản.

Nhiệm vụ hai, cách 1. Vẫn đối ngẫu theo cách trên, ta nhận được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P \leq 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Vì $Y_{i,j} \geq 0$, có thể thấy $P \in [-1, 1]$. Ý nghĩa của mô hình này thực chất là luồng chi phí lớn nhất trên đồ thị hai phía đầy đủ, chỉ khác ở chỗ ta có thể tăng giới hạn lượng ra của mọi đỉnh ở một phía thêm P , đồng thời giảm giới hạn lượng vào của mọi đỉnh phía còn lại đi P .

Trực giác cho thấy, khi P nằm trong các đoạn $[-1, 0]$ và $[0, 1]$, đáp án là một hàm đơn đỉnh. Tác giả ghi rằng việc chứng minh vẫn còn khó, nên trước hết thử lập bảng giá trị; kết quả cho thấy đúng là hàm đơn đỉnh. Gọi hàm đó là $f(P)$. Nó có một vài tính chất sau:

- Chỉ khi $f'(P) = 0$ mới có nhiều giá trị P cho cùng một giá trị $f(P)$. Vì vậy có thể tam phân hàm này.
- Với hàm đơn đỉnh trên $[-1, 1]$, chỉ cần thực hiện một lần tam phân.

Cách này giải được Task4.

Nhiệm vụ hai, cách 2. Đây là ý tưởng ban đầu của tác giả. Tuy độ phức tạp cao hơn, tư tưởng của nó khá khéo.

Nhận xét rằng đáp án của bài toán gốc có tính đơn điệu rõ ràng theo giá trị cần kiểm tra. Vì thế có thể nhị phân trực tiếp đáp án; giả sử đáp án đang thử là K . Khi đó ràng buộc ban đầu

$$\sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0$$

được thay bằng

$$\sum_{i=1}^N R_i = K, \quad \sum_{j=1}^N C_j = K.$$

Đối ngẫu tiếp, ta được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + A' \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + A' \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} + B' \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} + B' \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} + B' \leq 1, \\ Y_{i,j} \geq 0, \end{cases}$$

$$\max z = KA' + KB' + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Mô hình luồng của quy hoạch tuyến tính này cũng rất rõ: có thể dùng chi phí K để nối giới hạn lưu lượng của mọi đỉnh ở một phía thêm K . Theo tính chất của đối ngẫu, khi và chỉ khi mô hình này có nghiệm cực đại, bài toán gốc không có nghiệm, tức là giá trị K đang nhị phân nhỏ hơn đáp án đúng.

Rõ ràng khi mô hình này đạt nghiệm cực đại thì phải có $A' \rightarrow -\infty$ và $B' \rightarrow -\infty$. Nhưng ta không thể trực tiếp tìm A' và B' trên khoảng $(-\infty, 1]$. Có thể dự đoán rằng khi A' và B' đủ nhỏ, với một A' bất kỳ, cứ giảm A' đi ΔA thì lượng B' cần giảm, ΔB , sẽ tiến tới một hằng số.

Giả sử lúc này đã biết A' và B' . Nếu $\Delta A \geq \Delta B$, đặt $\Delta A = 1$, rồi tìm ΔB tương đối với ΔA . Rõ ràng ΔB nằm trong đoạn $[0, 1]$ và có thể tam phân: nếu ΔB quá nhỏ thì ΔA còn dư; nếu ΔB quá lớn thì ΔA không đủ bù chi phí $-\Delta B \cdot K$. Trường hợp $\Delta A \leq \Delta B$ làm tương tự bằng cách đặt $\Delta B = 1$.

Còn cần xác định A' và B' ban đầu. Vì A'/B' cuối cùng sẽ tiến tới $\Delta A/\Delta B$, chỉ cần thay $\Delta A = 1$ bằng $\Delta A = P_0$, với P_0 đủ lớn. Khi K càng gần đáp án, độ chính xác cần thiết của tỉ số A'/B' càng cao; nếu muốn nhận được chính xác đáp án K , P_0 phải rất lớn. Tuy nhiên đáp án chỉ cần 4 chữ số sau dấu thập phân, nên không cần chính xác đến vậy; P_0 có thể chọn tương đối nhỏ. Qua thử nghiệm, trong tình huống thông thường, lấy $P_0 = N^2$ là đủ để đạt nghiệm tối ưu.

So với thuật toán thứ nhất, cách này có thêm một lớp nhị phân, nên độ phức tạp tăng thêm một thừa số log. Đáng tiếc là nó chỉ vừa đủ qua dữ liệu $N \leq 50$, tức Task3.

4.2.5 Task2

Trong nhiệm vụ hai, với phần lớn dữ liệu ngẫu nhiên, $P = 0$ đã là nghiệm tối ưu. Chỉ có rất ít bộ dữ liệu làm P dao động trong khoảng $[-10^{-4}, 10^{-4}]$, về cơ bản có thể bỏ qua. Do đó có thể xuất đáp án của nhiệm vụ một cho câu hỏi thứ hai.

4.2.6 Khác

Nếu ma trận có kích thước $N \times M$ thì đương nhiên cũng có thể làm tương tự. Khi đó với nhiệm vụ một, P không còn bằng 0, mà bằng

$$P = \frac{M - N}{N + M}.$$

Bài này là một đồ thị hai phía đầy đủ. Nếu dùng SPFA để chạy luồng chi phí thì hiệu quả khá thấp; cần dùng zkw hoặc nguyên thủy-đối ngẫu.

Đơn hình trong nhiều trường hợp cũng có hiệu suất không tệ, gần với zkw luồng chi phí. Nhưng nếu người ra dữ liệu tận dụng đủ tri thức của mình thì vẫn có thể làm nó bị kẹt.

Đồ thị hai phía của nhiệm vụ hai có tính chất đặc biệt nhất định. Nếu có thể bỏ được bước tam phân, hoặc dùng cách khác để tối ưu mỗi lần chạy luồng chi phí, tác giả rất hoan nghênh trao đổi thêm.

4.3 Tổng kết

Bài này có nhiều phiên bản dẫn xuất. Tuy nhiên các phiên bản khác hoặc có đề bài quá phức tạp, hoặc tác giả chưa tìm được thuật toán đủ tốt. Nếu có ý tưởng tốt hơn cho lớp bài này, tác giả rất mong được liên hệ.

Cảm hứng của bài đến từ một bài trước đây của tác giả tên là *Chessboard*. Mô hình ban đầu là đặt quân xe trên bàn cờ, yêu cầu mỗi ô chỉ được đặt nhiều nhất một quân, và ô (i, j) cần được ít nhất $W(i, j)$ quân xe khống chế. Mô hình này tuy mô tả đơn giản, nhưng dường như là NP-khó: nó yêu cầu nghiệm nguyên của một quy hoạch tuyến tính, trong khi nghiệm tối ưu của quy hoạch tuyến tính này hầu như đều là số thực. Sau nhiều lần làm yếu ràng buộc, tác giả cuối cùng thu được bài toán hiện tại.

Bài toán kiểm tra kiến thức cơ bản của thí sinh về quy hoạch tuyến tính và mô hình hóa luồng mạng, đồng thời dùng nguyên lý đối ngẫu. Tính chất của thuật toán thứ nhất rất đẹp, dù phần chứng minh vẫn còn để ngỏ trong bài gốc. Với thuật toán thứ hai, ta lồng nhị phân và tam phân, rồi dùng luồng chi phí để phán định có nghiệm hay không. Tác giả cho rằng đây là một tư tưởng khá hay: chuyển bài toán tối ưu hóa thành bài toán phán định tính khả thi, rồi hiện thực nó trên mạng luồng. Tiếc rằng với bài này, đó không phải thuật toán tốt nhất.

Hy vọng bài toán giúp độc giả có thêm gợi mở và hiểu sâu hơn về quy hoạch tuyến tính cũng như các bài toán luồng mạng.

4.4 Lời cảm ơn

Cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi. Cảm ơn thầy Xiang Qizhong đã hướng dẫn tác giả. Cảm ơn các bạn học đã giúp đỡ tác giả.

5 Đa giác biến đổi

Dong Honghua, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết tổng kết các phép biến đổi thường dùng với đa giác, cho thấy tính đa dạng của cách nhìn một đa giác, đồng thời minh họa tác dụng của từng phép biến đổi qua ví dụ. Những biến đổi này cung cấp hướng suy nghĩ cho các bài toán liên quan tới đa giác. Bài viết cũng đi sâu vào một dạng biến đổi để nghiên cứu bài toán thống kê thông tin điểm bên trong đa giác, từ đó thu được một thuật toán có độ phức tạp khá tốt.

5.1 Dẫn nhập

Những năm gần đây, đa giác xuất hiện thường xuyên trong các kỳ chọn đội tuyển cấp tỉnh, các cuộc thi OI, cũng như ACM World Finals. Hình thức xuất hiện của chúng phức tạp và thay đổi đa dạng.

Tác giả tổng kết một số cách xử lý như sau. Phần 2 giới thiệu vài kiến thức cơ bản sẽ dùng về sau. Phần 3 liệt kê nhiều phép biến đổi của đa giác, kèm ví dụ để người đọc dễ hiểu và mở rộng. Phần 4 xuất phát từ GIS, tức hệ thống thông tin địa lý, để dẫn vào bài toán thống kê thông tin điểm bên trong đa giác; bài toán được thảo luận từ trường hợp đặc biệt tới tổng quát, từ dễ tới khó, và cuối cùng cho một thuật toán tổng quát có độ phức tạp tốt.

5.2 Cơ sở

5.2.1 Định nghĩa đa giác

Đa giác. Một hình phẳng khép kín tạo bởi ít nhất ba đoạn thẳng nối đuôi nhau theo thứ tự.

Đa giác đơn. Một đa giác mà các cạnh không kề nhau không giao nhau.

Đa giác lồi. Một đa giác đơn mà mọi góc trong đều không vượt quá 180° .

5.2.2 Tích có hướng

Với hai vector hai chiều $(x_1, y_1), (x_2, y_2)$, tích có hướng của chúng là

$$x_1y_2 - x_2y_1.$$

Giá trị này bằng diện tích có hướng của hình bình hành do hai vector tạo thành.

5.2.3 Phương pháp tia

Phương pháp tia dùng để phán định một điểm có nằm trong đa giác hay không. Từ điểm hiện tại, kẻ một tia theo một hướng nào đó rồi đếm số giao điểm giữa đa giác và tia này. Nếu số giao điểm là lẻ thì điểm nằm trong đa giác; nếu là chẵn thì điểm nằm ngoài.

Để tiện tính toán, có thể chọn tia hướng lên trên và hơi lệch sang phải, nhằm tránh trường hợp giao điểm rơi đúng vào đỉnh đa giác. Phương pháp này cũng áp dụng được cho đa giác tự cắt.

Ví dụ 1: Bean. Nguồn: [bzoj 1294] SCOI2009 Bean.

Đại ý đề bài. Cho $D \leq 9$ điểm có trọng số trên một lưới $n \times m$. Hãy tìm một đường đi xuất phát từ một điểm rồi quay lại điểm đó sao cho tổng trọng số các điểm bị bao quanh trừ đi số bước là lớn nhất.

Phân tích. Đường đi tạo thành một đa giác có thể tự cắt. Vì vậy việc phán định mỗi điểm có trọng số có bị bao quanh hay không trở thành bài toán điểm nằm trong đa giác.

Dùng phương pháp tia: với mỗi điểm có trọng số, kẻ tia hướng lên trên; khi đi qua một cạnh giao với tia đó thì trạng thái của điểm được lật. Dùng nhị phân để ghi trạng thái của từng điểm có trọng số, khi đó tổng giá trị điểm có thể suy ra từ trạng thái cuối.

Do đó chỉ cần liệt kê điểm xuất phát, tính đường ngắn nhất để đạt tới từng trạng thái nhị phân rồi quay lại điểm xuất phát. Vì trọng số cạnh đều bằng 1, có thể dùng bfs.

5.3 Các phép biến đổi

5.3.1 Kết hợp với gốc tọa độ

Cạnh là thành phần chính của đa giác. Nếu kết hợp mỗi cạnh với gốc tọa độ, ta thu được một số tam giác.

Theo cách phán định của phương pháp tia, có thể chứng minh diện tích đa giác bằng tổng diện tích có dấu của các tam giác này. Diện tích mỗi tam giác có thể tính trực tiếp bằng tích có hướng. Cần chú ý thứ tự lấy tích có hướng khác nhau sẽ cho dấu diện tích khác nhau; điều này vừa đúng lúc phản ánh chiều đi quanh của đa giác.

Ví dụ 2: Pollution Solution. Nguồn: *World Finals 2013 Problem J.*

Đại ý đề bài. Tính diện tích phần giao giữa một đa giác và một hình tròn.

Phân tích. Diện tích đa giác có thể tách thành một số diện tích có hướng của tam giác, nên bài toán rút gọn thành tính diện tích phần giao giữa tam giác và hình tròn. Nếu tách theo tâm đường tròn, ta bảo đảm một đỉnh của tam giác nằm ở tâm. Sau đó chia bốn trường hợp theo quan hệ vị trí giữa tam giác và đường tròn rồi tính riêng từng trường hợp.

5.3.2 Tách theo trục tọa độ

Tương tự, có thể kết hợp cạnh với trục tọa độ: trừ các cạnh thẳng đứng, mỗi cạnh được chiếu xuống một nửa trục, và cạnh cùng hình chiếu của nó tạo thành một hình thang vuông hoặc hình chữ nhật.

5.3.3 Nửa mặt phẳng

Một đa giác lồi có thể xem là giao của một số nửa mặt phẳng.

Ví dụ 3: Giao đa giác lồi. Nguồn: *[bzoj 2618] CQOI2006.*

Đại ý đề bài. Tính diện tích giao của n đa giác lồi.

Phân tích. Giao của các đa giác lồi chính là giao của toàn bộ các nửa mặt phẳng tương ứng.

Ở đây giới thiệu một thuật toán giao nửa mặt phẳng dùng phương pháp tăng dần, độ phức tạp bậc hai nhưng dễ viết. Với mỗi nửa mặt phẳng mới, tìm các điểm của giao hiện tại nằm bên trong và bên ngoài nửa mặt phẳng đó; xóa các điểm bên ngoài, đồng thời thêm giao điểm giữa nửa mặt phẳng mới và các đoạn nối một điểm trong với một điểm ngoài. Như vậy ta thu được giao mới.

5.3.4 Cây

Mọi đường chéo của đa giác lồi đều nằm bên trong nó, nên một đường chéo có thể chia đa giác lồi thành hai đa giác lồi. Làm đệ quy sẽ thu được một phép tam giác hóa. Các tam giác được tách ra tạo thành một cây.

Đa giác đơn cũng có thể biến thành cây. Từ mọi đỉnh, kéo dài theo phương ngang sang trái và phải cho tới giao điểm đầu tiên; kéo dài thành đường thẳng cũng được. Cách này chia đa giác thành nhiều hình thang, và các hình thang đó tạo thành một cây.

Ví dụ 4: Journey. Nguồn: *[bzoj 2657] ZJOI2012 journey.*

Đại ý đề bài. Cho phép tam giác hóa của một đa giác lồi. Hãy tìm một đường chéo sao cho số tam giác mà nó đi qua là lớn nhất.

Phân tích. Phép tam giác hóa tạo thành cấu trúc cây, nên thử biến đường chéo thành một đường đi trên cây.

Kết luận là: các đường chéo của đa giác lồi có số tam giác đi qua khác 0 tương ứng một-một với mọi đường đi trên cây.

Khi $n = 3$, kết luận hiển nhiên đúng. Giả sử kết luận đúng với $n = k$, ta chứng minh với $n = k + 1$. Xóa một đỉnh của đa giác lồi $k + 1$ cạnh thì thu được đa giác lồi k cạnh; trên cây chỉ cần xét trường hợp thêm một nút lá x , có cha là y . Theo giả thiết quy nạp, các đường chéo cũ tương ứng với các đường đi trong cây cũ. Khi thêm đỉnh bị xóa trở lại, vì y là cha của x , mọi đoạn nối từ đỉnh mới trong vùng x tới các đỉnh khác đều phải đi qua cạnh nối giữa x và y . Vì vậy các đoạn đó tương ứng với mọi đường đi trên cây xuất phát từ x . Kết luận đúng với $n = k + 1$.

Như vậy, tìm đường chéo đi qua nhiều tam giác nhất trở thành tìm đường dài nhất trên cây; có thể giải bằng quy hoạch động trên cây hoặc hai lần bfs.

Ví dụ 5: *Toil for Oil.* Nguồn: *World Finals 2002 Problem F.*

Đại ý đề bài. Cho một đa giác có lỗ trên biên. Hãy tính diện tích phần bên trong đa giác có thể chứa dầu.

Phân tích. Dùng các đường thẳng nằm ngang qua mọi đỉnh để cắt đa giác thành nhiều hình thang. Trong mỗi hình thang, hoặc toàn bộ có dầu, hoặc hoàn toàn không có dầu.

Quét từ dưới lên trên để sinh từng vùng. Có thể dùng đường trung tuyến của dải hiện tại để cắt: nếu nó giao với cạnh đa giác thì thêm biên trên và biên dưới; sắp xếp hai dãy biên này sẽ thu được quan hệ tương ứng. Nếu phía dưới thành phần liên thông hiện tại đã có lỗ, phần phía trên không thể có dầu, nên đánh dấu thành phần liên thông đó là không khả thi. Phần phía trên không ảnh hưởng tới dải hiện tại, nên trực tiếp cộng diện tích khả thi trong dải hiện tại vào đáp án.

Dùng DSU để duy trì tính liên thông của cây, đồng thời dùng các lỗ trên biên dưới trước khi thống kê dải hiện tại và trên biên trên sau khi thống kê để cập nhật trạng thái của thành phần liên thông.

5.4 Loạt *Architect*: thống kê thông tin điểm trong đa giác

5.4.1 Ứng dụng trong GIS

Trong GIS, hệ thống thông tin địa lý, ta thường cần thống kê các giá trị đặc trưng nằm trong một vùng. Vùng thường có thể biểu diễn bằng đa giác, nên các bài toán này trở thành bài toán thống kê thông tin điểm bên trong đa giác.

Trong phần lớn hệ thống GIS, cách làm là dùng phương pháp tia để lần lượt phán định từng điểm có nằm trong đa giác hay không. Thuật toán này có hiệu suất thấp và chưa tận dụng tốt các tính chất của bản đồ.

5.4.2 *Architect*

Đại ý đề bài. Nguồn: *POJ Challenge Round 5, lời mời của Trường Trung học số 1 Thiệu Hưng.*

Hỗ trợ thêm điểm động và hỏi số điểm nằm trong một đa giác đơn mà mọi góc trong đều là bội của 45° . Các điểm nằm ở tâm ô lưới, còn đa giác nằm trên biên ô lưới.

Vết cặn. Dùng phương pháp tia để lần lượt phán định từng điểm có nằm bên trong hay không.

Tách đa giác. Vì đa giác khá phức tạp, ta xét cách tách. Nếu đem phương pháp dùng gốc tọa độ và từng cạnh để tạo tam giác, vốn dùng khi tính diện tích đa giác, áp dụng vào bài này, ta sẽ sinh ra một số cạnh vừa không song song với trục tọa độ, vừa không tạo góc 45° ; điều này phá hỏng điều kiện của đề.

Vì vậy xét một cách tách khác. Trừ cạnh thẳng đứng, mỗi cạnh được chiếu xuống trục x ; cạnh đó và hình chiếu tạo thành hình thang vuông 45° hoặc hình chữ nhật. Số điểm trong toàn bộ các hình thang vuông 45° và hình chữ nhật, cộng trừ theo hướng duyệt, sẽ cho số điểm trong đa giác ban đầu: nếu duyệt theo chiều kim đồng hồ thì cạnh hướng sang phải lấy dấu $+$, cạnh hướng sang trái lấy dấu $-$. Cần chú ý rằng điểm nằm trên biên đa giác cũng được tính, nên hình thang vuông 45° mang dấu $-$ phải dịch xuống một ô.

Chuyển hóa bài toán. Sau khi tách đa giác, bài toán trở thành đếm số điểm trong các hình thang vuông 45° và hình chữ nhật.

Trước hết bỏ qua thao tác thêm điểm động; giả sử mọi điểm đã được thêm xong rồi mới hỏi. Với truy vấn số điểm trong một hình chữ nhật có một cạnh nằm trên trục x , có thể chuyển nó thành: phần đỏ cộng phần xanh, trừ phần xanh. Cả hai phần đều là truy vấn số điểm nằm phía dưới bên trái một điểm truy vấn.

Sau đó dùng thuật toán quét: sắp xếp thao tác thêm điểm và truy vấn theo tọa độ x , dùng tọa độ y làm khóa phụ; khi quét tới một truy vấn, đưa mọi điểm ở bên trái điểm truy vấn vào cấu trúc dữ liệu, chẳng hạn cây Fenwick, rồi theo tọa độ y để hỏi tổng đoạn hoặc tổng tiền tố trong cấu trúc dữ liệu.

Với hình thang vuông 45° , cũng dùng cách tương tự; lúc này khóa y được thay bằng $x + y$.

Chia để trị cdq. Cuối cùng, do đề bài cho thêm điểm và hỏi xen kẽ, còn đóng góp của điểm cho mỗi truy vấn là độc lập, có thể chia để trị theo thời gian, tức cdq. Tất cả sự kiện ban đầu được sắp theo thời gian; chọn trung điểm, chia dãy thành hai nửa, đệ quy trái và phải, rồi tính đóng góp của các điểm bên trái cho các truy vấn bên phải. Vì thời gian của bên trái đều nhỏ hơn bên phải, phần này trở lại trường hợp thêm điểm trước rồi mới hỏi, dùng phương pháp vừa nêu là được.

Độ phức tạp thời gian.

$$O(n \log^2 n).$$

5.4.3 Architekt1

Đại ý đề bài. Nguồn: lần hỏi thứ nhất của kỳ hồ tuyển đội tuyển 2014.

Ban đầu có n điểm trên mặt phẳng, mỗi điểm có trọng số. Có Q thao tác; mỗi thao tác là sửa trọng số điểm hoặc hỏi tổng trọng số các điểm nằm trong một đa giác đơn. Bảo đảm các cạnh của mọi đa giác truy vấn tạo thành một đồ thị phẳng liên thông.

Không có sửa đổi. Trước hết xét trường hợp không có sửa đổi. Tổng trọng số điểm có thể cộng trừ. Tách đa giác theo trục tọa độ: mỗi cạnh và hình chiếu của nó lên trục x tạo thành một hình thang. Ta cần tính tổng trọng số trong hình thang và cộng trừ theo hướng cạnh khi thống kê.

Chuyển sang cách nhìn “điểm đóng góp cho hình thang”. Dùng đường quét để tối ưu: dùng cây cân bằng duy trì các đoạn thẳng còn giao với đường quét; khi gặp một điểm ứng viên, cộng một giá trị cho mọi đoạn nằm phía trên nó. Cuối cùng thống kê tổng trọng số trên mọi đoạn và cộng trừ theo hướng.

Độ phức tạp thời gian.

$$O(S \log S),$$

trong đó S là tổng quy mô cạnh cần xử lý.

Chia để trị cdq. Nếu dùng thuật toán trên mà cần tăng hoặc giảm trọng số điểm thì làm thế nào?

Phần cộng trừ vẫn có thể tách được, nên có thể lồng cdq: biến tình huống sửa và hỏi xen kẽ thành sửa trước rồi hỏi sau. Cụ thể, với dãy hiện tại, chọn trung điểm, tính ảnh hưởng của các sửa đổi bên trái lên các truy vấn bên phải; phần này chính là trường hợp sửa trước hỏi sau, quay về bài toán trên. Sau đó đệ quy xử lý riêng nửa trái và nửa phải.

Độ phức tạp thời gian.

$$O(S \log S \log Q).$$

5.4.4 Archtext2

Đại ý đề bài. Nguồn: kỳ hồ tuyển đội tuyển 2014.

Trên cơ sở bài trước, ngoài tổng trọng số điểm còn phải hỏi trọng số điểm lớn nhất.

Chuyển hóa bài toán. Vì đã cho đồ thị phẳng, ta thử tìm cấu trúc trên đồ thị đối ngẫu. Trong đồ thị đối ngẫu, mỗi đa giác truy vấn bao quanh một thành phần liên thông, và thành phần đó cũng là liên thông trên đồ thị đối ngẫu. Mỗi cạnh của đa giác chính là một cạnh trên đồ thị đối ngẫu. Vì vậy bài toán chuyển thành: xóa một số cạnh rồi hỏi thông tin trong một thành phần liên thông. Khi đa giác suy biến thành đoạn thẳng thì bên trong không có miền nào; trường hợp này có thể đặc biệt xử lý trực tiếp.

Tiếp theo dẫn vào bài toán đồ thị động.

Bài toán đồ thị động. Mô tả ngắn gọn: nhiều lần hỏi thông tin của một thành phần liên thông sau khi xóa một số cạnh.

Dùng cdq. Tại mọi thời điểm, bảo đảm mọi cạnh không xuất hiện trong bất kỳ truy vấn nào của đoạn hiện tại đều đã được gộp. Đồng thời ghi số lần mỗi cạnh xuất hiện trong các truy vấn của đoạn.

Khi đệ quy sang nửa trái, thêm vào đồ thị các cạnh xuất hiện ở truy vấn nửa phải nhưng không xuất hiện ở truy vấn nửa trái, rồi dùng DSU để duy trì thông tin trong thành phần liên thông. Sau khi đệ quy trở về, cần khôi phục DSU về trạng thái trước khi đệ quy, tức xóa các cạnh vừa thêm do nửa trái.

Vì những cạnh bị xóa là các cạnh được thêm sau cùng, chỉ cần thao tác trên ngăn xếp thêm cạnh: hoàn tác thao tác gán cha trong DSU và khôi phục thông tin được duy trì. Khi đệ quy sang nửa phải cũng làm tương tự và khi quay lại cũng khôi phục.

Khi đoạn chỉ còn một truy vấn, trực tiếp lấy thông tin của thành phần liên thông chứa nó.

Tương ứng thông tin. Khó khăn còn lại là đưa thông tin điểm trong đồ thị phẳng ban đầu lên các đỉnh của đồ thị đối ngẫu, tức các miền của đồ thị phẳng.

Một truy vấn là phần bên trong của một đa giác. Cần tìm ít nhất một miền nằm bên trong, chẳng hạn miền ở bên phải một cạnh khi duyệt đa giác theo chiều kim đồng hồ, rồi hỏi thông tin của thành phần liên thông chứa miền đó.

Không có sửa đổi. Khi tính max, một điểm có thể bị tính lặp nhiều lần. Nếu không có sửa đổi, cứ lấy trọng số lớn nhất của mọi điểm thuộc một miền làm trọng số của miền đó, rồi áp dụng thuật toán đồ thị động để hỏi trọng số điểm lớn nhất trong một thành phần liên thông.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Số điểm. Xét tiếp trường hợp đặc biệt mọi trọng số điểm đều bằng 1, tức cần hỏi số điểm bên trong. Có thể dùng công thức Euler trên đồ thị phẳng liên thông:

$$\text{số điểm} = \text{số cạnh} - \text{số miền} - 1.$$

Số miền và số cạnh trong một thành phần liên thông có thể tính tương đối dễ, từ đó suy ra số điểm.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Trường hợp tổng quát. Với trường hợp tổng quát, ta cần ánh xạ mỗi điểm ban đầu vào một miền nào đó.

Tiếp tục phân tích truy vấn, có thể thấy lân cận của mọi điểm nằm bên trong đa giác cũng nằm trong thành phần liên thông tương ứng trên đồ thị đối ngẫu. Vì vậy, với điểm bên trong, chọn tùy ý một miền làm miền tương ứng là được.

Điểm trên biên không thỏa tính chất này, nhưng các điểm đó có thể lấy trực tiếp từ thông tin của truy vấn. Ngoài ra cần cưỡng chế rằng điểm trên biên không được thêm vào miền khi xử lý truy vấn; trong cdq, phải ghi thêm thông tin về số lần một điểm xuất hiện. Cụ thể, khi đệ quy nửa trái, cần đưa các điểm xuất hiện ở nửa phải nhưng không xuất hiện ở nửa trái vào miền tương ứng của chúng.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Thêm sửa đổi. Khi có thao tác sửa đổi, do ta đã có thông tin về số thao tác mà một điểm liên quan tới, chỉ cần tính cả sửa đổi vào đó. Khi đệ quy tới đoạn chỉ còn thao tác sửa đổi thì thực hiện sửa đổi.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

5.4.5 ArchiEXT

Đại ý đề bài. Trên cơ sở bài trước, không cho trước đồ thị phẳng liên thông tạo bởi toàn bộ đa giác truy vấn.

Tự lực xây dựng. Không cho đồ thị phẳng nhưng vẫn muốn dùng thuật toán trên, nên chỉ còn cách tự xây dựng đồ thị phẳng. Dùng đường quét để tìm mọi giao điểm giữa các cạnh của các đa giác; các giao điểm này cùng với các cạnh đa giác tạo thành đồ thị phẳng.

Bước này có độ phức tạp

$$O(\text{số giao điểm} \cdot \log S).$$

Không liên thông. Cần chú ý đồ thị phẳng thu được bằng cách này không nhất thiết liên thông. Vì vậy phải làm cho nó liên thông. Áp dụng phương pháp tác giả trình bày trong buổi trao đổi học viên WC2014, có thể chuyển nó thành đồ thị phẳng liên thông. Sau khi có đồ thị phẳng liên thông, bài toán trở thành bài trước.

Tối ưu. Điểm nghẽn của độ phức tạp là bước tìm toàn bộ giao điểm, tức $O(\text{số giao điểm})$. Tìm hết mọi giao điểm vừa không hợp lý lắm, vừa quá lãng phí.

Xét chia khối: cứ mỗi T truy vấn tạo thành một khối. Trong khối này cần dựng đồ thị phẳng; những điểm còn lại dùng định vị điểm bằng đường quét để đưa vào một miền. Khi đó dựng đồ thị phẳng tốn

$$O\left(\frac{S}{T}T^2 \log S\right),$$

định vị các điểm còn lại tốn

$$O\left(\frac{S}{T}S \log S\right),$$

và giải T truy vấn tốn

$$O\left(\frac{S}{T}T \log^2 T\right).$$

Tổng hợp lại, chọn $T = \sqrt{S}$ là tối ưu, cho độ phức tạp cuối cùng

$$O(S^{1.5} \log S).$$

Vẫn còn một vấn đề: nếu một đa giác truy vấn có quá nhiều cạnh, làm đây không thể chia theo $\sqrt{S}$ thì sao? Chú ý số đa giác như vậy không nhiều, và bên trong chỉ có một miền, nên có thể dùng trực tiếp định vị điểm để phán định, với tổng độ phức tạp $O((nQ + S) \log S)$. Lấy riêng mọi truy vấn có số cạnh lớn hơn $\sqrt{S}$, số lượng không vượt quá $\sqrt{S}$, rồi giải từng truy vấn độc lập; như vậy vẫn bảo đảm độ phức tạp $O(S^{1.5} \log S)$.

5.4.6 Tiểu kết

Khi các thuật toán thường dùng gặp khó ở truy vấn max, tác giả chọn một hướng khác: ghép nhiều đa giác thành một đồ thị phẳng rồi chuyển bài toán thành bài toán đồ thị.

Trong bài toán này còn nhiều lần dùng chia để trị theo thời gian, tức cdq, để biến phức tạp thành đơn giản. Ở cuối, bài viết dùng thêm tư tưởng chia khối và phân loại trường hợp để giảm độ phức tạp xấu nhất, từ đó thu được một thuật toán khá tốt.

5.5 Tổng kết

Mặc dù số cạnh của đa giác không cố định và hình dạng phức tạp, gây nhiều khó khăn khi giải bài, nhưng thông qua những cách tách khéo léo, đa giác có thể biến thành một số phần nhỏ hơn, tạo điều kiện để xử lý từng phần.

Sau khi phân tích đặc điểm của đa giác trong đề, áp dụng các phương pháp tách thường dùng thường đem lại hiệu quả tốt.

5.6 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Cảm ơn các thầy Chen Heli, Shao Hongxiang, You Guanghui và Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và hướng dẫn tác giả trong nhiều năm. Cảm ơn huấn luyện viên đội tuyển quốc gia Hu Weidong và Yu Linyun đã hướng dẫn. Cảm ơn các anh chị Xu Jie, Zhong Yiqin, Fei Jiezhong, Liang Jiawen của Đại học Thanh Hoa và Jiang Youzhi của Đại học Giao thông Thượng Hải đã giúp đỡ tác giả. Cảm ơn các bạn He Qizheng, Zheng Zhongyi, Xu Zhengjie của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn các bạn Yu Yili, Zhang Hengjie, Wang Jianhao, Guo Yu và nhiều bạn học khác của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn tất cả thầy cô và bạn bè từng giúp đỡ tác giả. Cảm ơn cha mẹ và gia đình đã ủng hộ, quan tâm và chăm sóc chu đáo.

Tài liệu tham khảo

1. SUN, “Bàn về một lớp thuật toán chia để trị”, bài giảng WC2013.
2. RRR, “Bàn về vài cách giải phi kinh điển cho bài cấu trúc dữ liệu”, luận văn đội tuyển quốc gia 2013.
3. HA, “Ứng dụng đường quét trong hình học tính toán”, trao đổi học viên WC2014.
4. 246, “Báo cáo ra đề *Architext*”, bài tập đội tuyển quốc gia 2014.

6 Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị

Cen Ruoxu, Trường Trung học Trần Hải, Chiết Giang

Tóm tắt

Nhóm hoán vị là một mô hình thường gặp trong các kỳ thi tin học. Bài viết này giới thiệu những kiến thức cơ bản về nhóm hoán vị, rồi xuất phát từ bài toán tổng quát là xác định một phần tử có thuộc nhóm hoán vị hay không để nghiên cứu sơ bộ thuật toán Schreier–Sims.

6.1 Kiến thức chuẩn bị

6.1.1 Nhóm

Giả sử G là một tập không rỗng, còn $\circ$ là một phép toán hai ngôi được định nghĩa trên G . Nếu thỏa mãn các điều kiện sau:

- tính đóng: $\forall a, b \in G, a \circ b \in G$;
- tính kết hợp: $\forall a, b, c \in G, (a \circ b) \circ c = a \circ (b \circ c)$;
- phần tử đơn vị: $\exists e \in G, \forall a \in G, e \circ a = a \circ e = a$;
- phần tử nghịch đảo: $\forall a \in G, \exists a^{-1} \in G, aa^{-1} = a^{-1}a = e$,

thì $(G, \circ)$ được gọi là một nhóm; viết gọn là G là nhóm. Từ đây trở đi, ta viết $a \circ b$ thành ab . Số phần tử trong một nhóm được gọi là bậc của nhóm; theo đó có nhóm hữu hạn và nhóm vô hạn. Nhóm là sự trừu tượng hóa của nhiều hệ đại số. Chẳng hạn, toàn bộ các số nguyên đối với phép cộng, hoặc các số nguyên trong $[1, n]$ nguyên tố cùng nhau với n đối với phép nhân modulo n , đều tạo thành nhóm. Cần chú ý rằng nhóm không nhất thiết thỏa mãn tính giao hoán.

Nếu $(G, \circ)$ là nhóm, H là tập con của G , và $(H, \circ)$ cũng là nhóm, thì H được gọi là nhóm con của G , ký hiệu $H \leq G$. Mọi nhóm G đều có hai nhóm con tầm thường là G và $\{e\}$; các nhóm con khác được gọi là nhóm con thực sự.

Với một tập con M của G , giao của tất cả các nhóm con chứa M cũng là một nhóm con; nhóm này được gọi là nhóm con sinh bởi M , ký hiệu $\langle M \rangle$. Có thể hiểu nhóm con sinh là nhóm thu được bằng cách lấy các phần tử trong M và thực hiện phép toán tùy ý nhiều lần. Khi đó M được gọi là một tập sinh của $\langle M \rangle$.

Ví dụ, xét nhóm đơn giản $G = \{0, 1, 2, 3, 4, 5\}$, phép toán là cộng modulo 6, phần tử đơn vị là 0. Khi đó $H_1 = \{0, 2, 4\}$ và $H_2 = \{0, 3\}$ là các nhóm con, trong đó H_1 cũng là nhóm con sinh bởi $\{2\}$.

6.1.2 Lớp kề

Nếu H là nhóm con của G , với mọi $a \in G$, gọi

$$aH = \{ah \mid h \in H\}$$

là một lớp kề trái của H , và gọi

$$Ha = \{ha \mid h \in H\}$$

là một lớp kề phải của H .

Trong G , số lớp kề trái của H bằng số lớp kề phải của H . Thật vậy, ánh xạ $Ha \mapsto a^{-1}H$ là một song ánh. Dưới đây ta chủ yếu xét lớp kề phải.

Trong ví dụ trên,

$$H_1 + 0 = H_1 + 2 = H_1 + 4 = \{0, 2, 4\},$$

$$H_1 + 1 = H_1 + 3 = H_1 + 5 = \{1, 3, 5\}.$$

Với H_2 , ta có $H_2 + 0 = H_2 + 3 = \{0, 3\}$, $H_2 + 1 = H_2 + 4 = \{1, 4\}$, và $H_2 + 2 = H_2 + 5 = \{2, 5\}$. Từ ví dụ này có thể quan sát được định lý sau.

Định lý 1. Nếu H là nhóm con của G , thì với mọi hai lớp kề phải Ha, Hb của H , hoặc $Ha = Hb$, hoặc $Ha \cap Hb = \emptyset$.

Chứng minh. Nếu tồn tại $x \in Ha \cap Hb$, thì tồn tại $h_1, h_2 \in H$ sao cho $x = h_1a = h_2b$. Do đó

$$\forall ha \in Ha, \quad ha = hh_1^{-1}h_1a = hh_1^{-1}h_2b = h'b \in Hb,$$

với $h' = hh_1^{-1}h_2 \in H$. Suy ra $Ha \subseteq Hb$; tương tự $Hb \subseteq Ha$. Vậy $Ha = Hb$. Định lý được chứng minh.

Định lý này cho thấy nhóm G có thể được chia thành hợp của các lớp kề phải đôi một rời nhau, từ đó cho ta một cách phân tích và giản hóa cấu trúc của nhóm.

Ký hiệu $|G : H|$ là số lớp kề phải phân biệt của nhóm con H trong G . Dùng 1 để chỉ nhóm con chỉ chứa phần tử đơn vị, thì $|G : 1|$ chính là bậc của G . Ta lập tức có định lý sau.

Định lý 2 (Lagrange). Nếu H là nhóm con của nhóm hữu hạn G , thì

$$|G : 1| = |G : H| |H : 1|.$$

6.1.3 Hoán vị

Một song ánh từ một tập hữu hạn Ω đến chính Ω được gọi là một hoán vị của Ω . Không mất tính tổng quát, coi $\Omega = \{1, 2, \dots, n\}$. Khi đó một hoán vị có thể biểu diễn dưới dạng

$$\begin{pmatrix} 1 & 2 & \cdots & n \\ a_1 & a_2 & \cdots & a_n \end{pmatrix},$$

trong đó $(a_1, a_2, \dots, a_n)$ là một hoán vị của dãy $1, 2, \dots, n$. Phép toán giữa các hoán vị là hợp thành:

$$\begin{pmatrix} 1 & 2 & \cdots & n \\ a_1 & a_2 & \cdots & a_n \end{pmatrix} \begin{pmatrix} 1 & 2 & \cdots & n \\ b_1 & b_2 & \cdots & b_n \end{pmatrix} = \begin{pmatrix} a_1 & a_2 & \cdots & a_n \\ b_1 & b_2 & \cdots & b_n \end{pmatrix}.$$

Trên n phần tử có $n!$ hoán vị. Để kiểm tra rằng $n!$ hoán vị này tạo thành một nhóm đối với phép hợp thành hoán vị, gọi là nhóm đối xứng bậc n , ký hiệu S_n . Nhóm hoán vị là nhóm con của S_n ; các phần tử của nó là các hoán vị.

Ảnh của phần tử β qua hoán vị g được ký hiệu là β^g . Tập các ảnh của β dưới mọi hoán vị trong nhóm hoán vị G được gọi là quỹ đạo của β , ký hiệu β^G . Trong nhóm hoán vị G , các hoán vị không làm thay đổi tập phần tử $A = \{a_1, \dots, a_m\}$ tạo thành nhóm con ổn định của A , ký hiệu G_A hoặc $G_{a_1, \dots, a_m}$.

6.1.4 Ví dụ 1: *Shift it!*

Tóm tắt đề bài. Có một lưới $n \times n$. Trên các ô lần lượt viết các số từ 1 đến n^2 , mỗi số xuất hiện đúng một lần. Mỗi thao tác có thể dịch vòng một hàng sang trái hoặc sang phải một ô, hoặc dịch vòng một cột lên trên hoặc xuống dưới một ô. Hãy xác định có thể dùng các thao tác đó để biến lưới ban đầu thành lưới đích hay không; nếu có thì hãy tìm một phương án bất kỳ không quá 10000 bước.

Quy mô dữ liệu. $n \leq 6$.

Phân tích. Tuy quy mô dữ liệu có vẻ rất nhỏ, số lượng dãy thao tác lại rất lớn nên khó tìm kiếm trực tiếp. Nhìn theo quan điểm nhóm hoán vị, ta thấy mỗi loại thao tác có thể biểu diễn thành một hoán vị trên n^2 số. Tập hợp tất cả thao tác có thể thực hiện chính là nhóm con G của S_{n^2} sinh bởi những hoán vị đó. Câu hỏi trở thành: hoán vị h biến lưới ban đầu thành lưới đích có thuộc G hay không.

Khi n là số chẵn, ta có thể dựng một phương án hoán đổi hai số kề nhau bất kỳ. Chẳng hạn, để hoán đổi 1 và 2 trong hình nguồn, lần lượt thực hiện một chuỗi dịch vòng trên hàng đầu và cột đầu: lên, trái, xuống, phải, lên. Hiệu quả của chuỗi thao tác này là hoán đổi 1 và 2, đồng thời các số còn lại trong cột đầu bị dịch vòng lên một ô. Lặp lại $n - 1$ lần thì cuối cùng chỉ còn tác dụng hoán đổi 1 và 2.

Vì có thể hoán đổi hai số kề nhau bất kỳ, hiển nhiên mọi lưới đích đều đạt được: ta chỉ cần lần lượt đưa từng số về vị trí cần đến bằng các phép hoán đổi. Kết hợp thêm vài chiến lược tham lam đơn giản có thể giảm số bước xuống trong giới hạn 10000.

Điểm thú vị là khi n lẻ thì không phải lúc nào cũng có lời giải. Ta biết một hoán vị có thể được tách thành tích của một số chu trình. Chu trình $(a_1, a_2, \dots, a_m)$ biểu thị a_1 chuyển thành a_2 , a_2 chuyển thành a_3 , ..., và a_m chuyển thành a_1 . Chu trình độ dài 2 được gọi là một phép đổi chỗ; mọi chu trình lại có thể tách thành tích của các phép đổi chỗ. Cách biểu diễn một hoán vị thành tích các phép đổi chỗ không duy nhất, nhưng tính chẵn lẻ là bất biến, từ đó chia hoán vị thành hoán vị lẻ và hoán vị chẵn. Khi n lẻ, mọi thao tác đều là hoán vị chẵn, nên không thể sinh ra hoán vị lẻ.

6.2 Thuật toán Schreier–Sims

6.2.1 Bài toán tổng quát về tính giải được của trò chơi hoán vị

Trong ví dụ 1, tuy ta thu được một kết luận đẹp, lời giải đã dùng một cấu trúc khó nghĩ ra và các tính chất rất đặc thù. Dưới đây ta xét bài toán tổng quát hơn: thực hiện thao tác trên n số, mỗi thao tác là một hoán vị, tập thao tác là S ; hỏi có thể biến một trạng thái thành một trạng thái khác hay không. Nói cách khác, cho $S \subseteq S_n$ và $h \in S_n$, cần xác định $h \in \langle S \rangle$ hay không. Dưới đây giả sử tập phần tử là $\Omega = \{1, 2, \dots, n\}$, và $G = \langle S \rangle \subseteq S_n$.

6.2.2 Cơ sở và tập sinh mạnh

Cơ sở của G là một dãy phần tử của Ω ,

$$B = (\beta_1, \dots, \beta_m),$$

thỏa $G_B = 1$, tức là trong G , hoán vị giữ nguyên mọi phần tử của B chỉ có thể là phần tử đơn vị. B định nghĩa một chuỗi nhóm con

$$G = G^{[1]} \geq G^{[2]} \geq \dots \geq G^{[m]} \geq G^{[m+1]} = 1. \quad (1)$$

Trong đó

$$G^{[i]} = G_{\beta_1, \dots, \beta_{i-1}},$$

tức là nhóm con gồm các hoán vị giữ nguyên $\{\beta_1, \dots, \beta_{i-1}\}$. Nếu với mọi $1 \leq i \leq m$ đều có $G^{[i+1]} \neq G^{[i]}$, thì cơ sở này được gọi là không dư thừa. Các cơ sở không dư thừa khác nhau có thể có độ dài khác nhau.

Một tập sinh T của nhóm G được gọi là tập sinh mạnh của G đối với cơ sở B , nếu

$$\forall 1 \leq i \leq m+1, \quad \langle T \cap G^{[i]} \rangle = G^{[i]}. \quad (2)$$

Nói cách khác, tập sinh mạnh phải chứa các tập sinh của tất cả nhóm con trong (1).

Ví dụ, xét $G = S_4$, dãy $B = (1, 2, 3)$ là một cơ sở không dư thừa của G . Ở đây $G^{[1]}, G^{[2]}, G^{[3]}$ lần lượt là tập tất cả hoán vị trên $\{1, 2, 3, 4\}$, trên $\{2, 3, 4\}$, và trên $\{3, 4\}$; $G^{[4]} = 1$, thỏa (1). Hai tập

$$T_1 = \{(1, 2, 3, 4), (3, 4)\}$$

và

$$T_2 = \{(1, 2, 3, 4), (2, 3, 4), (3, 4)\}$$

đều là tập sinh của G , nhưng T_1 không phải là tập sinh mạnh đối với B , vì $\langle T_1 \cap G^{[2]} \rangle \neq G^{[2]}$. Còn T_2 là một tập sinh mạnh đối với B .

6.2.3 Quá trình phân rã

Giả sử đã tìm được cơ sở B và tập sinh mạnh T . Ta sẽ dùng chúng để xác định h có thuộc G hay không.

Dùng các lớp kề của $G^{[i+1]}$ để chia $G^{[i]}$. Trong $G^{[i]}$, β_i có thể biến thành các phần tử thuộc quỹ đạo $\beta_i^{G^{[i]}}$, còn $G^{[i+1]}$ là nhóm ổn định của β_i . Do đó mỗi phần tử trong $\beta_i^{G^{[i]}}$ tương ứng với một lớp kề của $G^{[i+1]}$. Với mỗi lớp kề, ta chọn một phần tử đại diện r để biểu diễn lớp kề $G^{[i+1]}r$. Tập các phần tử đại diện này ký hiệu là R_i .

Có thể tìm R_i bằng BFS: lấy phần tử trong quỹ đạo làm đỉnh, lấy các hoán vị trong $T \cap G^{[i]}$ làm cạnh, và bắt đầu BFS từ β_i . Nếu đi được đến γ , điều đó chứng tỏ $\gamma \in \beta_i^{G^{[i]}}$. Nhân lần lượt các hoán vị trên đường đi từ β_i đến γ , ta thu được phần tử đại diện của lớp kề biến β_i thành γ .

Tiếp tục ví dụ trên, $G = S_4$, $B = (1, 2, 3)$, và

$$T = \{(1, 2, 3, 4), (2, 3, 4), (3, 4)\}.$$

Ta có $G^{[3]} = \{e, (3, 4)\}$. Nhóm này chia $G^{[2]}$ thành ba lớp kề:

$$G^{[3]}e = \{e, (3, 4)\},$$

$$G^{[3]}(2, 3) = \{(2, 3), (2, 3, 4)\},$$

$$G^{[3]}(2, 4) = \{(2, 4), (2, 3, 4)\}.$$

Do đó $R_2 = \{e, (2, 3), (2, 4)\}$. Các hoán vị trong ba lớp kề này lần lượt biến $\beta_2 = 2$ thành 2, 3, 4.

Bây giờ, mọi $g \in G$ có thể biểu diễn duy nhất dưới dạng

$$g = r_m r_{m-1} \dots r_1, \quad r_i \in R_i.$$

Quá trình phân rã rất đơn giản: với $g \in G$, trước hết tìm phần tử đại diện lớp kề $r_1 \in R_1$ thỏa $\beta_1^g = \beta_1^{r_1}$; sau đó đặt $g_2 = gr_1^{-1} \in G^{[2]}$. Tiếp tục tìm $r_2 \in R_2$ thỏa $\beta_2^{g_2} = \beta_2^{r_2}$, đặt $g_3 = g_2r_2^{-1}$, rồi lặp lại tương tự.

Quá trình phân rã có thể xác định h có thuộc G hay không. Ta thử phân rã h bằng phương pháp trên; nếu phân rã thành công thì hiển nhiên $h \in G$, ngược lại $h \notin G$. Có hai kiểu thất bại: hoặc trong quá trình phân rã, hoán vị

$$h_i = hr_1^{-1}r_2^{-1} \cdots r_{i-1}^{-1}$$

đưa β_i ra ngoài $\beta_i^{G^{[i]}}$, khiến ta không thể tiếp tục; hoán vị h_i này được gọi là hoán vị dư. Hoặc ở cuối quá trình, $h_{m+1} \neq e$; khi đó h_{m+1} cũng được gọi là một hoán vị dư.

6.2.4 Bổ đề Schreier

Nếu R là tập các phần tử đại diện lớp kề của H trong G , thì với mọi $g \in G$, $Hg \cap R$ chỉ có đúng một phần tử; ký hiệu phần tử đó là $\bar{g}$.

Định lý 3. Giả sử $H \leq G = \langle S \rangle$, R là tập các phần tử đại diện lớp kề của H trong G , và $e \in R$. Khi đó tập

$$T = \{rs\bar{r}s^{-1} \mid r \in R, s \in S\}$$

là một tập sinh của H .

Chứng minh. Theo định nghĩa, mọi phần tử trong T đều thuộc H , nên $\langle T \rangle \subseteq H$.

Lấy tùy ý $h \in H \leq G$. Viết $h = s_1s_2 \cdots s_k$, trong đó $s_i \in S$. Ta định nghĩa một dãy phần tử $h_0, h_1, \dots, h_k$ của G sao cho

$$h_j = t_1t_2 \cdots t_jr_{j+1}s_{j+1}s_{j+2} \cdots s_k. \quad (3)$$

Trong đó $t_i \in T$, $r_{j+1} \in R$, và $h_j = h$. Ban đầu đặt $h_0 = es_1s_2 \cdots s_k = h$. Nếu h_j đã được xác định, đặt

$$t_{j+1} = r_{j+1}s_{j+1}\overline{r_{j+1}s_{j+1}}^{-1}, \quad r_{j+2} = \overline{r_{j+1}s_{j+1}}.$$

Rõ ràng $h_{j+1} = h_j = h$, đúng theo dạng yêu cầu của (3). Ta có

$$h = h_k = t_1t_2 \cdots t_kr_{k+1}.$$

Do $h \in H$ và $t_1t_2 \cdots t_k \in \langle T \rangle \leq H$, suy ra $r_{k+1} \in H \cap R = 1$. Vì vậy $h \in \langle T \rangle$.

Tóm lại, $H \subseteq \langle T \rangle$, nên $\langle T \rangle = H$.

6.2.5 Quy trình thuật toán

Ta liên tục điều chỉnh và duy trì dãy phần tử

$$B = (\beta_1, \beta_2, \dots, \beta_m)$$

cùng dãy tập sinh $T_1, T_2, \dots, T_{m+1}$, bảo đảm T_i là tập ổn định của $\{\beta_1, \dots, \beta_{i-1}\}$, đồng thời $\langle T_i \rangle \geq \langle T_{i+1} \rangle$ với $1 \leq i \leq m$, $\langle T_1 \rangle = G$, và $T_{m+1} = 1$. Ta còn duy trì tập đại diện lớp kề R_i của $\langle T_i \rangle_{\beta_i}$ trong $\langle T_i \rangle$. Dùng một con trỏ cur , bảo đảm rằng khi $i > cur$ thì

$$\langle T_i \rangle_{\beta_i} = \langle T_{i+1} \rangle. \quad (4)$$

Khi đó $(\beta_{cur+1}, \dots, \beta_m)$ là một cơ sở của $\langle T_{cur} \rangle$, và

$$\bigcup_{cur+1 \leq j \leq m} T_j$$

là tập sinh mạnh của $\langle T_{cur} \rangle$.

1. Ban đầu đặt $m = 1$, chọn β_1 là một phần tử bất kỳ bị một hoán vị trong S làm thay đổi, đặt $T_1 = S$, $cur = 1$, rồi dùng BFS để tìm R_1 .
2. Mỗi lần xét xem (4) có đúng tại $i = cur$ hay không. Theo điều đã bảo đảm, $\langle T_{cur} \rangle_{\beta_{cur}} \supseteq \langle T_{cur+1} \rangle$, nên chỉ cần kiểm tra chiều ngược lại. Dùng định lý 3 để tìm tập sinh T' của $\langle T_{cur} \rangle_{\beta_{cur}}$, rồi dùng quá trình phân rã để xác định mọi phần tử của T' có thuộc $\langle T_{cur+1} \rangle$ hay không.
Nếu (4) đúng, giảm cur đi 1. Nếu không đúng, có một phần tử $h \in T'$ không thuộc $\langle T_{cur+1} \rangle$. Đưa hoán vị dư thu được khi phân rã h vào T_{cur+1} . Nếu $cur = m$, chọn một phần tử bất kỳ bị hoán vị dư này làm thay đổi làm β_{m+1} , rồi chạy lại BFS để cập nhật R_{cur+1} . Lúc này (4) tại $i = cur + 1$ chưa chắc đúng, nên tăng cur thêm 1.
3. Lặp lại quá trình trên đến khi $cur = 0$. Khi đó ta thu được cơ sở B của G và tập sinh mạnh

$$T = \bigcup_{1 \leq j \leq m} T_j.$$

6.2.6 Phân tích độ phức tạp

Kích thước của cơ sở nhiều nhất là n . Mỗi T_i nhiều nhất thay đổi n lần, vì sau mỗi lần thay đổi, quỹ đạo của β_i sẽ tăng thêm một phần tử. Vì vậy bước 2 được thực hiện $O(n^2)$ lần. Mỗi lần, kích thước tập sinh mạnh của $\langle T_{cur} \rangle_{\beta_{cur}}$ là

$$O(|R_{cur}| |T_{cur}|) = O(n^2),$$

và phân rã từng phần tử trong đó cần $O(n^2)$. Từ đó thu được cận trên thời gian $O(n^6)$.

Chú ý rằng nếu đã biết một phần tử nào đó của tập sinh mạnh của $\langle T_{cur} \rangle_{\beta_{cur}}$ thuộc $\langle T_{cur+1} \rangle$, về sau không cần phân rã nó lại nữa. Do đó, với mỗi $1 \leq cur \leq m$, chỉ cần thực hiện

$$O(|R_{cur}| |T_{cur}|) = O(n^2)$$

lần phân rã. Tổng độ phức tạp thời gian là $O(n^5)$. Nếu kích thước nhóm là $|G|$, độ phức tạp thời gian cũng có thể viết là $O(n^2 \log^3 |G|)$.

Trong thực tế, hằng số của thuật toán rất nhỏ, thường không cần đến nhiều thao tác như vậy. Chương trình của tác giả có thể chạy dữ liệu $n = 100$ trong 2 giây.

Độ phức tạp bộ nhớ là $O(n^3)$.

6.2.7 Ví dụ 2: Group

Tóm tắt đề bài. Cho một tập S gồm các hoán vị độ dài N , $|S| = M$. Hãy tìm bậc nhỏ nhất của một tập G thỏa các điều kiện:

$$S \subset G,$$

và

$$\forall \sigma, \tau \in G, \quad \sigma \tau^{-1} \in G.$$

Quy mô dữ liệu.

$$N \leq 50, \quad M \leq 100.$$

Phân tích. Trong G , tính kết hợp và phần tử nghịch đảo hiển nhiên đúng theo định nghĩa, tính đóng cũng đúng, và phần tử đơn vị $e = \sigma \sigma^{-1} \in G$. Do đó G là một nhóm. Nhóm nhỏ nhất chứa S chính là nhóm do S sinh ra. Vì vậy $G = \langle S \rangle$, và thứ cần tìm là $|\langle S \rangle|$.

Dùng thuật toán Schreier–Sims để tìm cơ sở B của $\langle S \rangle$. Ta biết B định nghĩa một chuỗi nhóm con

$$G = G^{[1]} \geq G^{[2]} \geq \dots \geq G^{[m]} \geq G^{[m+1]} = 1.$$

Theo định lý Lagrange,

$$|G| = \prod_{i=1}^m |G^{[i]} : G^{[i+1]}|.$$

Số lớp kề của $G^{[i+1]}$ trong $G^{[i]}$ chính là $|R_i|$. Thuật toán Schreier–Sims đã tìm được R_i , nên chỉ cần tính trực tiếp $\prod_{i=1}^m |R_i|$. Cần dùng số nguyên độ chính xác cao.

6.3 Tổng kết

Thuật toán Schreier–Sims là một thuật toán cơ bản trong nhóm tính toán. Còn có những thuật toán tương tự tốt hơn và các tối ưu cho thuật toán này, nhưng do trình độ của tác giả có hạn, đồng thời xét đến độ khó hiện thực trong OI, bài viết không giới thiệu thêm. Thuật toán này thực ra biểu diễn rõ ràng cấu trúc của nhóm hoán vị, nên có thể thuận tiện hoàn thành các nhiệm vụ như xác định tính thành viên và tính bậc. Tuy nhiên, đây là thuật toán tổng quát, không tận dụng tính chất đặc thù của một nhóm hoán vị cụ thể, nên độ phức tạp thời gian khá lớn. Khi đối mặt với bài cụ thể, ta vẫn cần khai thác tính chất riêng của đề rồi mới thiết kế thuật toán.

6.4 Lời cảm ơn

Cảm ơn bạn Du Yuhao đã giúp đỡ tác giả trong quá trình viết bài.

Cảm ơn thầy Li Jian đã hướng dẫn.

Tài liệu tham khảo

1. Akos Seress, *Permutation Group Algorithms*, Cambridge University Press.
2. Donald E. Knuth, “Efficient Representation of Perm Groups”, *Combinatorica* 11 (1991).
3. Nhóm biên soạn Khoa Toán ứng dụng, Đại học Đồng Tế, *Toán rời rạc*, Nhà xuất bản Đại học Đồng Tế.

7 Bàn về phụ thuộc tuyến tính

Kuang Zhengfei, Trường Yali, Trường Sa

Tóm tắt

Trong đời OI, rồi ở đại học hoặc những bậc học cao hơn, rất nhiều học sinh sẽ ít nhiều tiếp xúc với đại số tuyến tính. Phụ thuộc tuyến tính là một thành phần quan trọng của đại số tuyến tính. Bản thân nó không phức tạp, nhưng dấu vết của nó xuất hiện trong rất nhiều mảng kiến thức quan trọng. Bài viết này giới thiệu phụ thuộc tuyến tính, thảo luận các mở rộng của nó trong đại số tuyến tính, và một số ứng dụng kinh điển trong thi tin học.

7.1 Kiến thức chuẩn bị

Trước khi đi vào chủ đề chính, ta cần giới thiệu một ít kiến thức đại số tuyến tính.

7.1.1 Định thức của ma trận

Định thức là một hàm trong đại số tuyến tính, ánh xạ một ma trận vuông cấp n A tới một đại lượng vô hướng. Nó được định nghĩa bằng công thức toán học; bạn đọc có thể tra cứu tài liệu liên quan, ở đây không nhắc lại. Trong bài viết này, ta dùng $\det(A)$ hoặc $|A|$ để biểu thị định thức của ma trận A .

7.1.2 Ma trận chuyển vị

Trong đại số tuyến tính, chuyển vị của một ma trận $n \times m$ A là một ma trận khác, ký hiệu A^T . Mỗi hàng của A tương ứng với một cột của A^T . Ví dụ, với ma trận 3×2

$$\begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix},$$

chuyển vị của nó là

$$\begin{bmatrix} a & c & e \\ b & d & f \end{bmatrix}.$$

7.1.3 Ma trận nghịch đảo

Với một ma trận vuông cấp n A , nếu tồn tại một ma trận vuông cấp n khác B sao cho

$$AB = BA = I_n,$$

trong đó I_n là ma trận đơn vị cấp n , thì gọi A là khả nghịch, còn B là ma trận nghịch đảo của A .

Bổ đề 1. Một ma trận vuông A khả nghịch khi và chỉ khi $\det(A) \neq 0$ (quy tắc Cramer).

7.1.4 Không gian tuyến tính

Định nghĩa đầy đủ của không gian tuyến tính khá dài; ở đây ta chỉ giải thích không gian tuyến tính tạo bởi các vector k chiều, tức các bộ có thứ tự gồm k phần tử.

Giả sử V là một tập gồm các vector k chiều, còn $\mathbb{F}$ là một trường, thường là trường số. Nếu

$$\forall a, b \in V, \quad a + b \in V,$$

và

$$\forall a \in V, k \in \mathbb{F}, \quad ka \in V,$$

thì gọi V là một không gian tuyến tính trên trường $\mathbb{F}$. Cần đặc biệt chú ý rằng bất kể phần tử thật sự trong không gian tuyến tính là gì, ta đều gọi chúng là vector.

7.1.5 Tổ hợp tuyến tính

Giả sử V là một không gian tuyến tính trên trường $\mathbb{F}$, và

$$S = \{v_1, v_2, \dots, v_n\}$$

là một tập con của V . Với một phần tử $v \in V$, nếu tồn tại một bộ hệ số $\{a_1, a_2, \dots, a_n\} \in \mathbb{F}$ sao cho

$$v = a_1 v_1 + a_2 v_2 + \dots + a_n v_n,$$

thì gọi v là một tổ hợp tuyến tính của S .

7.2 Phụ thuộc tuyến tính

7.2.1 Phụ thuộc tuyến tính là gì

Gọi V là một không gian tuyến tính trên trường $\mathbb{F}$. Với một tập con

$$S = \{v_1, v_2, \dots, v_n\}$$

của V , nếu tồn tại một bộ hệ số không đồng thời bằng 0 $a_1, a_2, \dots, a_n$ trong $\mathbb{F}$ sao cho

$$a_1 v_1 + a_2 v_2 + \dots + a_n v_n = 0,$$

thì gọi S là phụ thuộc tuyến tính. Ở đây 0 là vector không. Ngược lại, nếu không tìm được bộ hệ số như vậy, thì gọi nhóm vector này là độc lập tuyến tính.

Nói cách khác, nếu vector không là một tổ hợp tuyến tính của S , thì gọi S phụ thuộc tuyến tính; nếu không thì gọi S độc lập tuyến tính. Một nhóm vector phụ thuộc tuyến tính được gọi là một nhóm phụ thuộc tuyến tính; trường hợp ngược lại tương tự.

Ví dụ, các vector $[1, 2, 0]$, $[0, -1, 1]$, $[1, 1, 1]$ phụ thuộc tuyến tính, vì vector cuối là tổng của hai vector đầu. Còn các vector $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$ độc lập tuyến tính.

Có một điểm cần nhắc tới: nếu trong công thức trên, ta thay toàn bộ phép toán bằng phép xor, thì cũng có thể nói rằng giữa chúng tồn tại một quan hệ tương tự. Trong OI, mối liên hệ giữa xor và phụ thuộc tuyến tính còn xuất hiện tương đối nhiều hơn.

7.2.2 Các định lý cơ bản về phụ thuộc tuyến tính

Phụ thuộc tuyến tính có nhiều định lý cơ bản.

Định lý 1. Giữa $k + 1$ vector k chiều nhất định có phụ thuộc tuyến tính.

Định lý 2. Mọi siêu tập của một nhóm phụ thuộc tuyến tính đều phụ thuộc tuyến tính.

Định lý 3. Mọi tập con của một nhóm độc lập tuyến tính đều độc lập tuyến tính.

Định lý 4. Nếu trong một nhóm vector S có một vector không, hoặc có hai vector bằng nhau, thì nhóm đó phụ thuộc tuyến tính.

Định lý 5. Mọi nhóm phụ thuộc tuyến tính đều có thể biểu diễn bằng tổ hợp tuyến tính của một nhóm độc lập tuyến tính.

Bản thân phụ thuộc tuyến tính không có quá nhiều điều đáng kể riêng lẻ, nhưng tần suất nó xuất hiện trong đại số tuyến tính lại rất cao. Đó là hướng mà ta sẽ thảo luận tiếp theo.

7.3 Cơ sở của không gian tuyến tính

Định nghĩa không gian tuyến tính đã được nhắc tới ở trên. Rõ ràng các không gian Euclid $\mathbb{R}^n$ với số chiều khác nhau đều là không gian tuyến tính trên trường số thực $\mathbb{R}$. Trong không gian Euclid hai chiều $\mathbb{R}^2$, mọi vector đều có thể được biểu diễn duy nhất bằng tổ hợp tuyến tính của hai vector độc lập tuyến tính $[0, 1]$, $[1, 0]$. Nếu mở rộng hiện tượng này, ta có thể nhận được tính chất tương tự cho mọi không gian tuyến tính. Vì vậy, ta gọi những vector tương tự $[0, 1]$, $[1, 0]$ là cơ sở của không gian tuyến tính. Định nghĩa chuẩn hơn là:

Giả sử V là một không gian tuyến tính, B là một tập con của V . Nếu mọi vector trong V đều có thể biểu diễn duy nhất thành tổ hợp tuyến tính của các vector trong B , thì gọi B là một cơ sở của V .

Từ trường hợp $\mathbb{R}^2$, không khó để nghĩ rằng có thể mọi cơ sở của không gian tuyến tính đều độc lập tuyến tính. Thực tế đúng là như vậy, vì nếu một cơ sở phụ thuộc tuyến tính thì sẽ có vô số cách tổ hợp tuyến tính để biểu diễn vector trong không gian. Vì thế, ta có thể mô tả cơ sở của không gian tuyến tính bằng một cách khác:

Cơ sở của một không gian tuyến tính là một tập con cực đại thỏa độc lập tuyến tính.

Hiển nhiên, một không gian tuyến tính có thể có nhiều cơ sở, nhưng số phần tử của chúng, dưới đây gọi là lực lượng cơ sở, đều bằng nhau; số đó được gọi là số chiều của không gian tuyến tính. Ngược lại, một nhóm độc lập tuyến tính chỉ có thể trở thành cơ sở của một không gian tuyến tính. Do đó, ta có thể dùng cơ sở để biểu diễn không gian tuyến tính. Tương tự, ta cũng có thể dùng một nhóm vector S , không nhất thiết độc lập tuyến tính, để xây dựng một không gian tuyến tính; không gian được xây dựng này gọi là $\text{span}(S)$.

Ngoài ra, cơ sở của một số không gian tuyến tính có kích thước vô hạn; ở đây chỉ nhắc qua và không trình bày kỹ. Bạn đọc có hứng thú có thể tìm đọc thêm tài liệu khác.

7.4 Hạng của ma trận

Ma trận có thể được xem như vector tạo bởi nhiều vector, lại có nhiều tính chất đẹp, nên giữ vị trí rất quan trọng trong đại số tuyến tính.

7.4.1 Hạng của ma trận là gì

Chính vì ma trận có thể được xem như một nhóm vector hàng hoặc vector cột, việc nghiên cứu các vector ấy có phụ thuộc tuyến tính hay không cũng là một vấn đề đáng xét. Hạng của ma trận xuất hiện từ đó.

Một ma trận A có hai loại hạng: hạng hàng và hạng cột. Hạng hàng là số lượng lớn nhất các hàng độc lập tuyến tính của ma trận A ; hạng cột là số lượng lớn nhất các cột thỏa điều kiện tương tự. Hiển nhiên, hạng hàng của A bằng hạng cột của A^T . Nhưng điều đẹp hơn là:

Định lý 6. Hạng hàng của một ma trận bằng hạng cột của nó.

Có rất nhiều cách chứng minh; ở đây chỉ đưa ra một cách. Tuy nhiên, trước khi chứng minh, ta còn cần thảo luận một vài thứ khác.

7.4.2 Quan hệ giữa hạng và cơ sở

Ma trận có hạng, còn không gian tuyến tính có cơ sở. Trước đó ta đã nhắc rằng một nhóm vector có thể xây dựng ra một không gian tuyến tính, còn ma trận vừa khéo chính là một nhóm vector như vậy. Không khó nhận ra rằng tập tương ứng với hạng hàng của ma trận chính là cơ sở của không gian tuyến tính tạo bởi các vector hàng của nó, dưới đây gọi là không gian hàng; hạng cột thì tương ứng với cơ sở của không gian cột.

7.4.3 Hạng hàng bằng hạng cột

Với thảo luận ở phần trước, ta có thể chứng minh định lý trên.

Xét một ma trận $n \times m$ A . Giả sử hạng hàng của nó là r , tức số chiều của không gian hàng V của A là r . Trong V , chọn tùy ý một nhóm vector

$$S = \{v_1, v_2, \dots, v_r\},$$

rồi dùng các vector này làm hàng để tạo thành một ma trận $r \times m$ R . Hiển nhiên, mỗi vector hàng của A đều có thể biểu diễn bằng một tổ hợp tuyến tính của S . Nói bằng phép nhân ma trận, nhất định tồn tại một ma trận $n \times r$ C sao cho

$$A = CR.$$

Vì $A = CR$, mỗi vector cột của A đều có thể biểu diễn bằng tổ hợp tuyến tính của các vector cột trong C . Rõ ràng không gian cột của A được chứa hoàn toàn trong không gian cột của C , nên hạng cột của A không lớn hơn hạng cột của C . Lại vì C chỉ có r cột, nên hạng cột của C không lớn hơn r , tức hạng hàng của A . Do đó hạng cột của A không lớn hơn hạng hàng của A . Theo tính đối xứng, hạng hàng của A cũng không lớn hơn hạng cột của A . Vì vậy hạng hàng của A bằng hạng cột của A . Chứng minh xong.

Vì hạng hàng bằng hạng cột, ta thống nhất hai khái niệm này và gọi chung là hạng của ma trận, ký hiệu $R(A)$. Hiển nhiên, với một ma trận $n \times m$ A ,

$$R(A) \leq \min(n, m).$$

Khi $R(A) = \min(n, m)$, ta gọi A là đầy hạng; ngược lại gọi là thiếu hạng.

7.4.4 Hạng và định thức

Khi định thức của một ma trận vuông cấp n A khác 0, không khó chứng minh n vector hàng của A độc lập tuyến tính. Nói cách khác, ma trận này đầy hạng. Vì thế, nếu dùng định thức để hiểu hạng, ta có một định nghĩa mới:

Khi trong một ma trận A tồn tại một định thức con cấp r , tức chọn tùy ý r vector hàng để lập ma trận rồi lấy định thức, khác 0, còn mọi định thức con cấp $r + 1$ đều bằng 0, ta quy định hạng của A là $R(A) = r$.

Đến đây, quan hệ giữa cơ sở, hạng và định thức đều đã được phân tích. Bây giờ chỉ còn một vấn đề: tính hạng như thế nào.

7.4.5 Cách tính hạng

Nếu cần tính hạng, phương pháp phổ biến nhất là khử Gauss. Tin rằng tất cả học sinh từng học OI đều đã viết khử Gauss, và xem đây là cách thông dụng để giải một hệ phương trình tuyến tính. Thực ra, sau khi khử Gauss một ma trận, hay nói cách khác một nhóm vector hàng, A , thứ thu được chính là một cơ sở trong $\text{span}(A)$. Nói cách khác, sau khi khử Gauss một lần, số phần tử khác 0 còn lại chính là $R(A)$.

Thông thường, độ phức tạp thời gian của khử Gauss là $O(n^3)$. Nhưng nếu thay tổ hợp tuyến tính giữa các vector bằng xor giữa các phần tử, và gọi P là cỡ lớn nhất của phần tử, thì số chiều của không gian tuyến tính tương ứng với một nhóm phần tử nhiều nhất là $O(\log P)$; khi đó độ phức tạp của khử Gauss trở thành $O(n \log P)$.

7.4.6 Tổng xor lớn nhất

Nhắc đến khử Gauss thì không thể không nhắc đến bài toán tổng xor lớn nhất. Đây là một bài toán tương đối kinh điển trong OI: cho một nhóm số tự nhiên, hãy tìm tập con có xor lớn nhất. Rõ ràng, sau khi khử Gauss theo xor nhóm số này, ta sẽ thu được một tam giác ngược, với các phần tử giảm dần từ trên xuống dưới. Khi đó, ta duyệt từng phần tử từ trên xuống dưới, lấy giá trị lớn hơn giữa đáp án hiện tại và xor của nó với phần tử đang xét làm đáp án mới. Không khó chứng minh đáp án sinh ra theo cách này nhất định là tối ưu.

7.5 Liên hệ ngoài đại số tuyến tính

Các chương trước đều nói về một vài mở rộng của phụ thuộc tuyến tính trong đại số tuyến tính, thiên về toán học hơn. Thực ra phụ thuộc tuyến tính cũng có liên hệ với rất nhiều thứ khác.

7.5.1 Phụ thuộc tuyến tính và matroid

Nhiều bạn từng học tham lam đều biết matroid, tức nguồn gốc của cách chứng minh thuật toán tham lam. Dưới đây, ta dùng cặp $M = \{S, L\}$ để biểu diễn một matroid. Giả sử A là một nhóm vector hàng, có thể chứng minh:

Định lý 7. Lấy tập nền S là nhóm vector A , và lấy L là họ mọi tập con độc lập tuyến tính của A ; khi đó (S, L) cấu thành một matroid.

Matroid có bốn tính chất. Nếu cả bốn tính chất đều được thỏa, định lý trên được chứng minh. Trong đó hai tính chất là hiển nhiên, chỉ cần chứng minh tính di truyền và tính trao đổi.

Tính di truyền. Suy ra từ Định lý 3.

Tính trao đổi. Dùng phản chứng. Giả sử $X, Y \in L$ và $|X| > |Y|$. Nếu với mọi $v \in X - Y$ đều có $Y \cup \{v\} \notin L$, thì mọi vector trong X đều có thể biểu diễn bằng tổ hợp tuyến tính của các vector trong Y . Nhưng X độc lập tuyến tính, không thể được biểu diễn hoàn toàn bằng một tập có số phần tử nhỏ hơn; điều này mâu thuẫn với $|X| > |Y|$. Vì vậy tính trao đổi nhất định được thỏa. Chứng minh xong.

Với tính chất này, dùng tham lam là có thể giải bài toán nhóm độc lập tuyến tính có trọng số lớn nhất. Ta xem tiếp bài sau.

7.5.2 CQOI2013 Problem 1 Nim

Tóm tắt đề bài. Hiện có hai người A, B chơi một trò chơi. Luật chơi như sau.

Trong trò chơi có N đồng diêm. Ở lượt đầu tiên, A lấy ra một số đồng diêm, nhưng không được lấy hết; sau đó B cũng thực hiện thao tác tương tự. Tiếp theo, nếu xor số que diêm trong các đồng còn lại bằng 0, thì B thắng; ngược lại A thắng.

Hỏi A cần lấy ít nhất bao nhiêu que diêm để có thể thắng.

Quy mô dữ liệu.

$$N \leq 100, \quad \text{mỗi số que diêm} \leq 10^9.$$

Phân tích. Nếu sau khi A lấy một số đồng diêm, các phần tử còn lại độc lập tuyến tính, thì A thắng.

Vì vậy bài toán trở thành: cho một số tự nhiên, cần chọn một tập con độc lập tuyến tính sao cho tổng mọi số trong tập con là lớn nhất. Theo thảo luận trước đó, dùng tham lam là có thể giải được.

7.5.3 Phụ thuộc tuyến tính và đồ thị

Nếu biểu diễn một đồ thị vô hướng bằng ma trận kề điểm-cạnh, thì một nhóm độc lập tuyến tính trong ma trận, tức nhóm vector, tương ứng với một rừng khung trong đồ thị.

Đi sâu hơn một chút, nếu ta có thể tính được một nhóm vector có bao nhiêu nhóm độc lập tuyến tính, thì có thể giải bài toán đếm số rừng khung trong một đồ thị. Không may là bài sau là một bài toán #P, tương tự vai trò của NP trong các bài toán tối ưu, nên bài trước chắc chắn cũng là bài toán #P. Nếu bạn đọc có hứng thú với hướng này, có thể thử tự khám phá.

7.5.4 Phụ thuộc tuyến tính và hình học tính toán

Trong $\mathbb{R}^2$, điều kiện cần và đủ để hai vector đồng tuyến là giữa chúng có phụ thuộc tuyến tính. Trong $\mathbb{R}^3$, điều tương ứng là quan hệ đồng phẳng, và cứ thế suy rộng. Nếu quan sát kỹ, ta có thể thấy tính chất này tương đương với việc dùng tích có hướng, tức lấy định thức, để phán đoán.

7.6 Bài ví dụ

Tục ngữ nói thực hành mới sinh hiểu biết. Để thể hiện tốt hơn vai trò của đại số tuyến tính trong OI, dưới đây liệt kê vài bài ví dụ về phụ thuộc tuyến tính.

7.6.1 XXor, bài tự ra

Tóm tắt đề bài. Định nghĩa một hàm $f(S)$, ánh xạ một tập số tự nhiên tới một số. Nếu trong S có tổng cộng k tập con sao cho xor của mọi số trong tập con ấy bằng 0, thì $f(S) = k^2$. Cho một tập L gồm n số tự nhiên, hãy tính

$$\sum_{T \subset L} f(T).$$

Quy mô dữ liệu.

$$n \leq 24, \quad \text{mọi phần tử trong } S \text{ đều } \leq 10^{18}.$$

Phân tích. Không khó nhận ra $f(S)$ thực chất tương ứng với

$$2^{2(|S| - \dim \text{span}(S))}.$$

Chỉ cần biết số chiều của mọi không gian tuyến tính là bài toán được giải.

Trước đó đã nhắc rằng, với một nhóm phần tử, độ phức tạp của khử Gauss theo xor là $O(n \log P)$, trong đó P là cỡ lớn nhất của phần tử. Bài này có thể khử Gauss theo xor trực tiếp trên mọi tập, nhưng tổng độ phức tạp vẫn quá chậm. Ta liên tưởng tới việc dùng hàm `lowbit` để tính số bit 1 trong mỗi xâu nhị phân độ dài n trong thời gian $O(2^n)$; bài này cũng có thể dùng cách tương tự.

Với mỗi tập S , ta dùng một danh sách liên kết để ghi lại dãy sinh bởi cơ sở của $\text{span}(S)$ sau khi sắp theo kích thước. Ta duyệt S theo số phần tử từ lớn đến nhỏ. Nếu $S = \emptyset$, lực lượng cơ sở của nó là 0. Ngược lại, chọn tùy ý một phần tử $x \in S$, dùng cơ sở của $\text{span}(S - \{x\})$ từ lớn đến nhỏ để lần lượt khử x , tức nếu có thể làm x nhỏ hơn thì xor nó. Nếu cuối cùng $x = 0$, thì

$$\text{span}(S) = \text{span}(S - \{x\});$$

ngược lại, cơ sở của $\text{span}(S)$ bằng cơ sở của $\text{span}(S - \{x\})$ hợp với x sinh ra cuối cùng, rồi lưu bằng danh sách liên kết.

Đến đây bài toán được giải. Đặt P là số lớn nhất trong L ; độ phức tạp thời gian và bộ nhớ đều là $O(2^n \log P)$.

7.6.2 Codeforces Round #228 Div.1 D

Tóm tắt đề bài. Gọi S là một tập trên miền số tự nhiên $\mathbb{Z}_+$. Nếu với mọi $a, b \in S$ đều có

$$a \oplus b \in S,$$

thì gọi S là một tập hoàn hảo.

Cho một số n , hỏi có bao nhiêu tập hoàn hảo sao cho số lớn nhất trong tập không vượt quá n .

Quy mô dữ liệu.

$$n \leq 10^9.$$

Phân tích. Để đơn giản hóa đề bài, ta xem mọi số trong bài này là số 32 bit. Khi đó một số thực chất tương ứng với một vector 32 chiều.

Không khó nhận ra một tập hoàn hảo thực chất tương ứng với một không gian tuyến tính; ta có thể dùng cơ sở để biểu diễn nó. Nói cách khác, nếu với mỗi không gian tuyến tính thỏa điều kiện, ta đều có thể tìm ra một cơ sở tương ứng duy nhất, thì bài toán được giải.

Dựa trên ý tưởng khử Gauss, ta có thể biểu diễn cơ sở tương ứng với một tập hoàn hảo bằng một tam giác ngược, hoặc hình thang. Lại vì phần tử lớn nhất trong tập không được vượt quá n , ta có thể dùng quy hoạch động chữ số, xử lý từng bit từ cao xuống thấp, lần lượt thêm các vector cơ sở.

Trước hết bỏ qua ràng buộc không lớn hơn n . Khi đang xử lý bit thứ i , và đã thêm j vector cơ sở:

1. Nếu thêm một vector cơ sở mới 2^i , thì các bit thứ i của những vector trước đó có thể được tùy ý không chế, nên có thể đặt tất cả bằng 0. Khi đó có 1 cách chọn.
2. Nếu không thêm vector mới, có thể chứng minh rằng dù bit thứ i của các vector trước đó lấy giá trị thế nào, không gian tuyến tính tương ứng đều không bị đếm lặp. Khi đó có 2^j cách chọn.

Dùng DP là có thể giải được bài toán, nhưng lúc này còn phải xét ràng buộc không lớn hơn n .

Đặt $g_{i,j}$ là số không gian tuyến tính có hậu tố i bit của giá trị lớn nhất bằng hậu tố i bit của n , sau khi đã xử lý i bit thấp và sinh ra j vector cơ sở. Tương tự, đặt $f_{i,j}$ là số không gian tuyến tính có hậu tố i bit của giá trị lớn nhất nhỏ hơn hậu tố i bit của n . Có thể thấy quá trình chuyển của $f_{i,j}$ hoàn toàn giống quá trình trên; ta chỉ cần xét cách chuyển $g_{i,j}$. Khi xử lý bit thứ i và đang có j vector cơ sở:

1. Nếu bit thứ i của n bằng 1:

- Nếu thêm vector cơ sở mới 2^i , thì hậu tố của giá trị lớn nhất trong không gian tuyến tính vẫn bằng hậu tố của n .
- Nếu không thêm, có 2^{j-1} cách gán để hậu tố vẫn bằng nhau, và cũng có 2^{j-1} cách để hậu tố nhỏ hơn n .

2. Nếu không, ta không thể thêm lại một vector, và chỉ có 2^{j-1} cách gán để hậu tố vẫn bằng nhau. Chú ý trường hợp đặc biệt $j = 0$ cũng có 1 cách.

Trên đây là toàn bộ quá trình chuyển. Đáp án của bài này là

$$\sum_{i=0}^{32} g_{0,i} + f_{0,i}.$$

Độ phức tạp thời gian là $O(\log^2 n)$, độ phức tạp bộ nhớ là $O(\log^2 n)$.

7.6.3 HEOI2013 Canxi sắt kẽm selen vitamin

Tóm tắt đề bài. Cho n vector n chiều, gọi là loại A , rồi cho tiếp n vector n chiều, gọi là loại B . Bảo đảm các vector loại A độc lập tuyến tính. Cần chọn cho mỗi vector loại A một vector dự phòng từ các vector loại B , sao cho sau khi vector ấy bị thay bằng vector dự phòng, n vector mới vẫn độc lập tuyến tính. Hai vector loại A bất kỳ không được chọn cùng một vector dự phòng. Hãy tìm một phương án như vậy.

Quy mô dữ liệu.

$$n \leq 300, \quad \text{mọi số xuất hiện đều là số nguyên trong } [0, 10000].$$

Phân tích. Gọi tập các vector loại A là S_A , và tập các vector loại B là S_B .

Bài toán thực chất là hỏi giữa mỗi cặp vector A, B có thể thay thế được hay không; phần còn lại chỉ cần dùng ghép cặp đồ thị hai phía. Hiển nhiên $\text{span}(S_A) = \mathbb{R}^n$, mọi vector loại B đều có thể được biểu diễn duy nhất bằng một tổ hợp tuyến tính của S_A . Trên thực tế có thể chứng minh:

Định lý 8. Điều kiện cần và đủ để một vector loại B x có thể thay một vector loại A y là: trong tổ hợp tuyến tính biểu diễn x bằng các phần tử của S_A , hệ số của y khác 0.

Chứng minh. Điều này là hiển nhiên. Nếu hệ số của y khác 0, thì y có thể được biểu diễn bằng x và các phần tử khác của S_A . Nói cách khác, sau khi thay thế vẫn có $\text{span}(S_A) = \mathbb{R}^n$, và S_A độc lập tuyến tính. Ngược lại, nếu hệ số bằng 0, các phần tử khác trong S_A có thể trực tiếp tổ hợp ra x , nên các vector này nhất định phụ thuộc tuyến tính. Chứng minh xong.

Bây giờ bài toán trở thành cách tính các hệ số trong tổ hợp tuyến tính tương ứng với x . Có thể giải phần này bằng một phương pháp tương tự khử Gauss.

Đến đây bài toán được giải. Độ phức tạp thời gian của thuật toán là $O(N^3)$, độ phức tạp bộ nhớ là $O(N^2)$. Chú ý rằng đây không phải độ phức tạp của lời giải gốc của bài.

7.7 Tổng kết

Tuy phụ thuộc tuyến tính không xuất hiện quá thường xuyên trong OI, vẫn có rất nhiều bài hay liên quan tới nó. Khác với số học đang phổ biến hiện nay, các bài về phụ thuộc tuyến tính thường kết hợp với một số thứ ngoài toán học. Bởi vì bản thân nó không phức tạp, nhưng yêu cầu thí sinh có năng lực hiểu và phân tích tương đối mạnh thì mới vận dụng đến nơi đến chốn. Phụ thuộc tuyến tính là một chủ đề đẹp; hy vọng trong tương lai OI sẽ có thêm nhiều bài liên quan tới nó, để nó được ứng dụng nhiều hơn.

7.8 Lời cảm ơn

- Cảm ơn cha mẹ và nhà trường đã bồi dưỡng tác giả.
- Cảm ơn CCF đã trao cho tác giả cơ hội quý báu này.
- Cảm ơn thầy Qu Yunhua đã hỗ trợ và hướng dẫn tác giả.
- Cảm ơn các bạn Liu Yanyi, Huang Zhi'ao và những bạn khác đã giúp đỡ tích cực.

Tài liệu tham khảo

1. Wikipedia, <http://www.wikipedia.org>.
2. Khoa Toán, Đại học Đồng Tế, *Đại số tuyến tính*, Nhà xuất bản Đại học Đồng Tế.
3. XFS, *Bước đầu nghiên cứu về matroid*, Tập luận văn đội tuyển quốc gia năm 2007.
4. Heidi Gebauer, Yoshio Okamoto, “Fast Exponential-Time Algorithms for the Forest Counting and the Tutte Polynomial Computation in Graph Classes”.

Ghi chú về nguồn

Tệp PDF có 228 trang. pdftotext đọc tốt phần mục lục nhưng phần thân chủ yếu trở thành ký tự mojibake do ánh xạ phonetic. Bài MSS được dịch từ các trang in 1–8 (trang vật lý PDF 5–12). Bài Matrix được dịch từ các trang in 9–18 (trang vật lý PDF 13–22); trang in 18 là trang trắng ngăn cách, nội dung bài nằm trên các trang in 9–17. Bài Đa giác biến đổi được dịch từ các trang in 19–34 (trang vật lý PDF 23–38); trang in 34 là trang trắng ngăn cách, nội dung bài nằm trên các trang in 19–33. Các bài đã dịch đều dùng các trang render bằng pdftoppm và OCR bằng tesseract. Một số ký hiệu trong phần ràng buộc nhóm D của bài MSS bị nhiễu, nên bản dịch ghi rõ phần suy ra từ lời giải thay vì chép một công thức không chắc chắn. Ở bài Matrix, các công thức đối ngẫu được đối chiếu giữa OCR, lớp văn bản mojibake còn đọc được ký hiệu và ảnh render của các trang nguồn. Ở bài Đa giác biến đổi, các hình minh họa trong PDF nguồn được chuyển thành mô tả văn bản; các dòng độ phức tạp bị OCR nhiễu được đối chiếu lại bằng ảnh render. Bài Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị được dịch từ các trang in 35–44 (trang vật lý PDF 39–48); trang in 44 là trang trắng ngăn cách, nội dung bài nằm trên các trang in 35–43. Các công thức về cơ sở, tập sinh mạnh, bổ đề Schreier và độ phức tạp được đối chiếu lại bằng ảnh render vì OCR thường nhầm ký hiệu lớp kề và chỉ số. Bài Bàn về phụ thuộc tuyến tính được dịch từ các trang in 45–56 (trang vật lý PDF 49–60). Công thức ma trận và các chuyển trạng thái trong ví dụ Codeforces được đối chiếu lại bằng ảnh render; phần ký hiệu f, g trong chuyển trạng thái được dịch theo vai trò được mô tả ngay trước công thức đáp án vì bản nguồn có một vài chỗ dễ nhầm giữa hai chữ này.